



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Solving Navier-Stokes equations for the "flow past a cylinder" problem

Tobia Grimaldi, 11127377
Luca Salmoiraghi, 10849129
Dong Hua Zhu, 10827613

Advisor:

Prof. Alfio Maria Quarteroni

Co-advisors:

Michele Bucelli

Academic year:

2025-2026

Abstract: This research introduces a finite element solver for the incompressible Navier-Stokes equations. Developed using C++ deal.II library, the solver employs stable $\mathbb{P}^2/\mathbb{P}^1$ elements for space discretisation and the θ -method for time discretisation. The investigation focuses on capturing steady and unsteady laminar flow patterns in the classical CFD benchmark "flow past a cylinder", evaluating performance of the solver through drag and lift coefficients analysis.

Key-words: Navier-Stokes equations, Finite Element Method (FEM), Flow past a cylinder, deal.II library, CFD benchmark

1. Introduction

This document presents an overview of the most prominent finite element methods for solving the incompressible Navier-Stokes equations to simulate the 2D/3D benchmark problem "flow past a cylinder" for different values of the Reynolds number $Re \leq 200$.

This first section is intended to be a brief introduction to the physical and mathematical problem of interest, in particular with respect to the equations that model the physical situation and weak formulation of such equations.

1.1. Introduction to Navier-Stokes equations

The celebrated Navier-Stokes equations describe the motion of a fluid, whose density ρ is constant, in a domain $\Omega \subset \mathbb{R}^d$ (with $d = 2, 3$). In the most general version, they read as follows [3]:

$$\begin{cases} \frac{\partial \underline{u}}{\partial t} - \operatorname{div} [\nu (\nabla \underline{u} + \nabla \underline{u}^T)] + (\underline{u} \cdot \nabla) \underline{u} + \nabla p = \underline{f} & \underline{x} \in \Omega, t > 0 \\ \operatorname{div}(\underline{u}) = 0 & \underline{x} \in \Omega, t > 0 \end{cases} \quad (1)$$

with $\underline{u} = \underline{u}(\underline{x}, t)$ being the velocity of the fluid, ν the kinematic viscosity, p the pressure divided by the density (which will be addressed simply as "pressure" from now on) and $\underline{f}$ a forcing term per unit of mass, which is supposed to be an element of $L^2(\mathbb{R}^+; [L^2(\Omega)]^d)$ (to say, for each time $t \in \mathbb{R}^+$, each component of $\underline{f}$ is in $L^2(\Omega)$). Each of the two equations has a precise meaning: the second represents conservation of mass (the so called "continuity equation"), whilst the first is the conservation of linear momentum, where the term

$-\operatorname{div} [\nu (\nabla \underline{u} + \nabla \underline{u}^T)]$ describes molecular diffusion, while $(\underline{u} \cdot \nabla) \underline{u}$ convective transport. When ν is constant, from the continuity equation it is possible to obtain the following:

$$\operatorname{div} [\nu (\nabla \underline{u} + \nabla \underline{u}^T)] = \nu (\Delta \underline{u} + \nabla \operatorname{div}(\underline{u})) = \nu \Delta \underline{u}$$

Therefore, the set of equations (1) can be rewritten in the equivalent form:

$$\begin{cases} \frac{\partial \underline{u}}{\partial t} - \nu \Delta \underline{u} + (\underline{u} \cdot \nabla) \underline{u} + \nabla p = \underline{f} & \underline{x} \in \Omega, t > 0 \\ \operatorname{div}(\underline{u}) = 0 & \underline{x} \in \Omega, t > 0 \end{cases} \quad (2)$$

Such equations (2) are known as **incompressible Navier-Stokes equations**, which are the ones that will be considered from now on.

Of course, in order for the equations (2) to be well posed, both initial and boundary conditions must be provided. Consider therefore $\Gamma_D; \Gamma_N \subseteq \partial\Omega$ such that $\Gamma_D \cap \Gamma_N = \emptyset$ and $\Gamma_D \cup \Gamma_N = \partial\Omega$. The required conditions can be expressed as follows:

$$\begin{cases} \underline{u}(\underline{x}, t = 0) = \underline{u}_0(\underline{x}) & \underline{x} \in \Omega \\ \underline{u}(\underline{x}, t) = \underline{g}(\underline{x}, t) & \underline{x} \in \Gamma_D \\ \left(\nu \frac{\partial \underline{u}}{\partial \hat{\eta}} - p \hat{\eta} \right)(\underline{x}, t) = \underline{h}(\underline{x}, t) & \underline{x} \in \Gamma_N \end{cases} \quad (3)$$

where $\hat{\eta}$ is the normal outward unitary vector to $\partial\Omega$.

Therefore, this paper will discuss how to numerically solve the equations (2) with initial and boundary conditions (3).

Moreover, if the term

$$(\underline{u} \cdot \nabla) \underline{u} = 0$$

the Navier-Stokes equations yield to a linear formulation which is called unsteady Stokes equations (see 2.2.1). In general, to study whether such a term is negligible or not, the so called **Reynolds number** $\mathbb{Re}$ is introduced:

$$\mathbb{Re} := \frac{|\underline{U}| L}{\nu} \quad (4)$$

where L is the length of the computational domain, ν is once again the viscosity of the fluid and $|\underline{U}|$ is the modulus of the so-called characteristic speed of the fluid. If $\mathbb{Re}$ is sufficiently small (in general, if $\mathbb{Re} = O(1)$, but this is very application-dependent), Stokes equations approximate well the behaviour of such a fluid.

1.2. Weak formulation of the Navier-Stokes equations

The weak formulation for (2) with initial and boundary conditions (3) (where, for simplicity, $\underline{g} = \underline{0} \quad \forall \underline{x} \in \Gamma_D$ will be considered) can be obtained as follows:

- Consider a generic test function φ belonging to a functional space $\mathcal{V}$ (specified further on). Multiply the first of the two equations of (2) by φ and integrate over Ω :

$$\int_{\Omega} \frac{\partial \underline{u}}{\partial t} \cdot \varphi d\Omega - \int_{\Omega} \nu \Delta \underline{u} \cdot \varphi d\Omega + \int_{\Omega} [(\underline{u} \cdot \nabla) \underline{u}] \cdot \varphi d\Omega + \int_{\Omega} \nabla p \cdot \varphi d\Omega = \int_{\Omega} \underline{f} \cdot \varphi d\Omega \quad (5)$$

As a result of the Green's formulae, the followings relations hold:

$$\begin{aligned} * \quad & - \int_{\Omega} \nu \Delta \underline{u} \cdot \varphi d\Omega = \int_{\Omega} \nu \nabla \underline{u} \cdot \nabla \varphi d\Omega - \int_{\partial\Omega} \nu \frac{\partial \underline{u}}{\partial \hat{\eta}} \cdot \varphi d\gamma, \\ * \quad & \int_{\Omega} \nabla p \cdot \varphi d\Omega = - \int_{\Omega} p \operatorname{div}(\varphi) d\Omega + \int_{\partial\Omega} p \varphi \cdot \hat{\eta} d\gamma. \end{aligned}$$

and by applying such relations in (5), it is possible to obtain:

$$\begin{aligned} \int_{\Omega} \frac{\partial \underline{u}}{\partial t} \cdot \varphi d\Omega + \nu \int_{\Omega} \nabla \underline{u} \cdot \nabla \varphi d\Omega + \int_{\Omega} [(\underline{u} \cdot \nabla) \underline{u}] \cdot \varphi d\Omega - \int_{\Omega} p \operatorname{div}(\varphi) d\Omega = \\ = \int_{\Omega} \underline{f} \cdot \varphi d\Omega + \int_{\partial\Omega} \left(\nu \frac{\partial \underline{u}}{\partial \hat{\eta}} - p \hat{\eta} \right) \cdot \varphi d\gamma \quad \forall \varphi \in \mathcal{V} \end{aligned} \quad (6)$$

- In a similar way, consider a test function ψ belonging to a functional space $\mathcal{Q}$ (yet to be defined), multiply the second equation of (2) and integrate over Ω , in order to obtain:

$$\int_{\Omega} \psi \operatorname{div}(\underline{u}) d\Omega = 0 \quad \forall \psi \in \mathcal{Q} \quad (7)$$

There remains to discuss the choice of the two functional spaces $\mathcal{V}$ and $\mathcal{Q}$:

- $\mathcal{V}$ is chosen in such a way that the test function vanishes on Γ_D , to say:

$$\mathcal{V} = \left[H_{\Gamma_D}^1(\Omega) \right]^d = \left\{ \underline{\varphi} \in [H^1(\Omega)]^d \mid \underline{\varphi}|_{\Gamma_D} = 0 \right\}$$

where $H^1(\Omega) \equiv W^{1,2}(\Omega)$, where $W^{1,2}(\Omega)$ represents the Sobolev 1, 2 space.

- The choice of $\mathcal{Q}$ is more delicate. Since p is never derived in (2) and as a consequence neither is ψ , there is no need to consider Sobolev spaces for defining $\mathcal{Q}$. If $\Gamma_N \neq \emptyset$ then it is sufficient to consider $\mathcal{Q} = L^2(\Omega)$. On the other hand, if $\Gamma_N = \emptyset$, then the pressure appears in (2) exclusively in terms of its gradient, hence if $\tilde{p}$ is a solution of the equations, then $(\tilde{p} + c)$ with $c \in \mathbb{R}$ would be a solution as well, since clearly $\nabla \tilde{p} = \nabla(\tilde{p} + c)$. To avoid such undetermined behaviour, it is customary to require the solution $\tilde{p}$ to have zero average, that is $\tilde{p} \in L_0^2(\Omega)$, where:

$$L_0^2 := \left\{ p \in L^2(\Omega) \mid \int_{\Omega} p \, d\Omega = 0 \right\}$$

This choice reflects on $\mathcal{Q}$, hence if $\Gamma_N = \emptyset$, then $\mathcal{Q} = L_0^2(\Omega)$.

Therefore, to summarise:

$$\mathcal{Q} = \begin{cases} L^2(\Omega) & \text{if } \Gamma_N \neq \emptyset \\ L_0^2(\Omega) & \text{if } \Gamma_N = \emptyset \end{cases}$$

To sum up, the weak formulation of (2) and (3) is the following:

Find $\underline{u} \in L^2(\mathbb{R}^+; [H^1(\Omega)]^d) \cap C^0(\mathbb{R}^+; [L^2(\Omega)]^d)$ and $p \in L^2(\mathbb{R}^+; \mathcal{Q})$ such that:

$$\begin{cases} \int_{\Omega} \frac{\partial \underline{u}}{\partial t} \cdot \underline{\varphi} \, d\Omega + \nu \int_{\Omega} \nabla \underline{u} \cdot \nabla \underline{\varphi} \, d\Omega + \int_{\Omega} [(\underline{u} \cdot \nabla) \underline{u}] \cdot \underline{\varphi} \, d\Omega - \int_{\Omega} p \operatorname{div}(\underline{\varphi}) \, d\Omega = \\ = \int_{\Omega} \underline{f} \cdot \underline{\varphi} \, d\Omega + \int_{\partial\Omega} \left(\nu \frac{\partial \underline{u}}{\partial \hat{\eta}} - p \hat{\eta} \right) \cdot \underline{\varphi} \, d\gamma \quad \forall \underline{\varphi} \in \mathcal{V} \\ \int_{\Omega} \psi \operatorname{div}(\underline{u}) \, d\Omega = 0 \quad \forall \psi \in \mathcal{Q} \end{cases} \quad (8)$$

2. Finite Elements Method for the Navier-Stokes equations

2.1. Galerkin weak formulation

Before proceeding further, a discretised version of (8) must be obtained. This is accomplished using the standard Galerkin weak formulation. Introduce therefore:

- $a(\underline{u}; \underline{\varphi}) := \nu \int_{\Omega} \nabla \underline{u} \cdot \nabla \underline{\varphi} \, d\Omega$
- $c(\underline{u}, \underline{z}, \underline{v}) := \int_{\Omega} [(\underline{u} \cdot \nabla) \underline{z}] \cdot \underline{v} \, d\Omega$
- $b(\underline{\varphi}; p) := - \int_{\Omega} p \operatorname{div}(\underline{\varphi}) \, d\Omega$
- $F(\underline{\varphi}) := \int_{\Omega} \underline{f} \cdot \underline{\varphi} \, d\Omega + \int_{\partial\Omega} \left(\nu \frac{\partial \underline{u}}{\partial \hat{\eta}} - p \hat{\eta} \right) \cdot \underline{\varphi} \, d\gamma$

which allow to write the variational weak formulation for (2) and (3) (in the case of $\underline{g} = \underline{0} \quad \forall \underline{x} \in \Gamma_D$) in the following form:

Find $\underline{u} \in L^2(\mathbb{R}^+; [H^1(\Omega)]^d) \cap C^0(\mathbb{R}^+; [L^2(\Omega)]^d)$ and $p \in L^2(\mathbb{R}^+; \mathcal{Q})$ such that:

$$\begin{cases} \int_{\Omega} \frac{\partial \underline{u}}{\partial t} \cdot \underline{\varphi} \, d\Omega + a(\underline{u}; \underline{\varphi}) + b(\underline{\varphi}; p) + c(\underline{u}; \underline{u}; \underline{\varphi}) = F(\underline{\varphi}) & \forall \underline{\varphi} \in \mathcal{V} \\ b(\underline{u}; \psi) = 0 & \forall \psi \in \mathcal{Q}_h \end{cases} \quad (9)$$

To achieve the desired discretisation, let $\mathcal{V}_h \subset \mathcal{V}$ be a finite-dimensional subspace, with $\dim(\mathcal{V}_h) = N_\varphi \in \mathbb{N}^+$, and let $\mathcal{Q}_h \subset \mathcal{Q}$ be a finite-dimensional subspace, with $\dim(\mathcal{Q}_h) = N_q \in \mathbb{N}^+$. The Galerkin formulation for the variational weak formulation (9) is:

Find $\underline{u} \in (\mathbb{R}^+; \mathcal{V}_h)$ and $p \in (\mathbb{R}^+; \mathcal{Q}_h)$ such that:

$$\begin{cases} \int_{\Omega} \frac{\partial \underline{u}_h}{\partial t} \cdot \underline{\varphi}_h d\Omega + a(\underline{u}_h; \underline{\varphi}_h) + b(\underline{\varphi}_h; p_h) + c(\underline{u}_h; \underline{u}_h; \underline{\varphi}_h) = F(\underline{\varphi}_h) & \forall \underline{\varphi}_h \in \mathcal{V}_h \\ b(\underline{u}_h; \psi_h) = 0 & \forall \psi_h \in \mathcal{Q}_h \end{cases} \quad (10)$$

2.2. Inf-Sup condition and stability analysis

Before proceeding any further, it is necessary to assess the existence and stability of the solution of the discretised problem (10).

2.2.1 Stokes equations

To better isolate and analyse the stability issue, the steady, linear Stokes equations will be considered in this and the next paragraph. Such equations can be obtained from (2) by neglecting time dependence and the convective term, thus leading to:

$$\begin{cases} -\nu \Delta \underline{u} + \nabla p = \underline{f} & \underline{x} \in \Omega \\ \operatorname{div}(\underline{u}) = 0 & \underline{x} \in \Omega, t > 0 \end{cases} \quad (11)$$

whose corresponding discrete Galerkin formulation is:

Find $\underline{u} \in \mathcal{V}_h$ and $p \in \mathcal{Q}_h$ such that:

$$\begin{cases} a(\underline{u}_h; \underline{\varphi}_h) + b(\underline{\varphi}_h; p_h) = F(\underline{\varphi}_h) & \forall \underline{\varphi}_h \in \mathcal{V}_h \\ b(\underline{u}_h; \psi_h) = 0 & \forall \psi_h \in \mathcal{Q}_h \end{cases} \quad (12)$$

where $a(\cdot; \cdot)$ and $b(\cdot; \cdot)$ are the bilinear forms defined in 2.1.

2.2.2 Babuška-Brezzi theorem and Inf-Sup condition

The main theoretical result concerning the existence of a solution for the problem (12) is known as Babuška-Brezzi theorem [1]. Its statement is the following:

Theorem 2.1. *Let $(\mathcal{V}; \|\cdot\|_{\mathcal{V}})$ and $(\mathcal{Q}; \|\cdot\|_{\mathcal{Q}})$ be two Hilbert spaces. Consider:*

- $a : \mathcal{V} \times \mathcal{V} \rightarrow \mathbb{R}$ bilinear form such that a is continuous, to say $\exists \gamma > 0$ such that:

$$|a(u; v)| \leq \gamma \|u\|_{\mathcal{V}} \|v\|_{\mathcal{V}} \quad \forall u; v \in \mathcal{V}$$

and a is coercive, to say $\exists \alpha > 0$ such that:

$$a(v; v) \geq \alpha \|v\|_{\mathcal{V}}^2 \quad \forall v \in \mathcal{V}$$

- $b : \mathcal{V} \times \mathcal{Q} \rightarrow \mathbb{R}$ bilinear such that b is continuous, to say $\exists \delta > 0$ such that:

$$|b(u; w)| \leq \delta \|u\|_{\mathcal{V}} \|w\|_{\mathcal{Q}} \quad \forall u \in \mathcal{V} \quad \forall w \in \mathcal{Q}$$

and satisfies the following condition, also known as the Ladyzhenskaya–Babuška–Brezzi (LBB) condition or inf-sup condition: $\exists \beta > 0$ independent of the mesh size h such that $\forall q_h \in \mathcal{Q}_h \exists \underline{v}_h \in \mathcal{V}_h$ where:

$$\inf_{q_h \in \mathcal{Q}_h - \{0\}} \left\{ \sup_{\underline{v}_h \in \mathcal{V}_h - \{0\}} \left\{ \frac{b(\underline{v}_h; q_h)}{\|\underline{v}_h\|_{\mathcal{V}} \|q_h\|_{\mathcal{Q}}} \right\} \right\} \geq \beta \quad (13)$$

Then $\exists! \underline{u}_h \in \mathcal{V}_h$ and $\exists! p_h \in \mathcal{Q}_h$ solution to the Galerkin formulation of the Stokes equations (12). Moreover, the following stability conditions hold:

$$\begin{cases} \|\underline{u}_h\|_{\mathcal{V}} \leq \frac{1}{\alpha} \|F\|_{\mathcal{V}'} \\ \|p_h\|_{\mathcal{Q}} \leq \frac{1}{\beta} \left(1 + \frac{\gamma}{\alpha}\right) \|F\|_{\mathcal{V}'} \end{cases}$$

where $\mathcal{V}'$ is the (topological) dual space of $\mathcal{V}$ and $\|\cdot\|_{\mathcal{V}'}$ its norm.

Among all hypothesis of the Babuška-Brezzi theorem, the LBB condition (13) is particularly meaningful when choosing the finite elements spaces $\mathcal{V}_h$ and $\mathcal{Q}_h$. Indeed, the inf-sup condition can be interpreted as a compatibility condition between the space of velocity and that of pressure, to say that a correctly chose $\mathcal{V}_h$ should be rich enough relative to $\mathcal{Q}_h$. To better understand its importance, suppose there exists at least one node where (13) is violated, therefore on such node $\exists p_h^* \in \mathcal{Q}_h$ such that $b(\underline{\varphi}_h; p_h^*) = 0 \quad \forall \underline{\varphi}_h \in \mathcal{V}_h$. This is problematic, since, by the linearity of $b(\cdot; \cdot)$, $b(\underline{v}_h; p + c \cdot p_h^*) = b(\underline{v}_h; p)$ with $c \in \mathbb{R}$, hence on such a node there would exist infinitely many pressure solutions, leading to a loss of uniqueness and stability. In the literature, such points where the LBB condition fails are called parasitic nodes.

2.3. Taylor-Hood elements

A popular and robust choice for the finite elements $\mathcal{V}_h$ and $\mathcal{Q}_h$ is the family of Taylor-Hood elements [2]. Given a mesh τ_h and denoting a generic element by K , the corresponding spaces are defined as follows:

$$\begin{aligned} \bullet \mathcal{V}_h &= \left\{ \underline{\varphi}_h \in [C^0(\Omega)]^d \mid \underline{\varphi}_h|_K \in [\mathbb{P}^{r+1}]^d; \underline{\varphi}_h|_{\Gamma_d} = 0 \quad \forall K \in \tau_h \right\} \\ \bullet \mathcal{Q}_h &= \left\{ \psi_h \in \mathcal{Q} \mid \psi_h|_K \in \mathbb{P}^r \cap C^0(\Omega) \quad \forall K \in \tau_h \right\} \end{aligned}$$

where r is the degree of the polynomial basis for $\mathcal{Q}_h$ and $r+1$ that of the polynomial basis of $\mathcal{V}_h$. Taylor-Hood elements satisfy the LBB condition, therefore the discrete problem is well posed and free of parasitic pressure nodes. Moreover, the following convergence estimate for such elements holds:

Theorem 2.2. Assume $\underline{u} \in [H^{r+2}(\Omega)]^d$ and $p \in H^{r+1}(\Omega)$. Then there exists $c > 0$, independent of h , such that:

$$\|\underline{u} - \underline{u}_h\|_{\mathcal{V}} + \|p - p_h\|_{\mathcal{Q}} \leq c \cdot h^{r+1} \left[\|\underline{u}\|_{[H^{r+2}(\Omega)]^d} + \|p\|_{H^{r+1}(\Omega)} \right]$$

2.4. Time discretisation and fully discrete problem

To obtain a fully discretised problem, a discretisation in time must be performed as well. To simplify the notation, from now on the following assumptions will be considered:

- $\Gamma_D = \partial\Omega$, that is $\Gamma_N = \emptyset$;
- $\underline{g} = \underline{0} \quad \forall \underline{x} \in \Gamma_D = \partial\Omega$

Let now $\{\underline{\xi}_i\}_{i=1}^{N_\varphi}$ and $\{\chi_i\}_{i=1}^{N_q}$ be finite elements basis for $\mathcal{V}_h$ and $\mathcal{Q}_h$ respectively. $\underline{u}_h$ and p_h can therefore be rewritten as:

$$\underline{u}_h(t; \underline{x}) = \sum_{j=1}^{N_\varphi} u_j(t) \underline{\xi}_j(\underline{x}) \quad \wedge \quad p_h(t; \underline{x}) = \sum_{k=1}^{N_q} p_k(t) \chi_k(\underline{x})$$

Introduce:

$$\underline{u}(t) := [u_1(t) \quad \dots \quad u_{N_\varphi}(t)]^T \quad \wedge \quad \underline{p}(t) := [p_1(t) \quad \dots \quad p_{N_q}(t)]^T$$

and define the following matrices and vectors:

- the mass matrix M , where:

$$M_{ij} = \int_{\Omega} \underline{\xi}_j \cdot \underline{\xi}_i \, d\Omega$$

- the viscous and diffusion matrix A , where:

$$A_{ij} = a(\underline{\xi}_j; \underline{\xi}_i)$$

- the convection matrix $C(\underline{u})$, where:

$$C(\underline{u})_{ij} = \sum_{k=1}^{N_\varphi} u_k c(\underline{\xi}_k; \underline{\xi}_j; \underline{\xi}_i)$$

- the divergence matrix B , where:

$$B_{ij} = -b(\underline{\xi}_j; \chi_i)$$

- the right-hand side vector $\underline{\mathcal{F}}(t)$, where:

$$\mathcal{F}_i(t) = \int_{\Omega} \underline{f} \cdot \underline{\xi}_i d\Omega$$

The system of equation (10) can therefore be rewritten as:

$$\begin{cases} M \frac{d\underline{u}(t)}{dt} + A\underline{u}(t) + C[\underline{u}(t)] - B^T \underline{p}(t) = \underline{\mathcal{F}}(t) \\ B\underline{u}(t) = \underline{0} \end{cases} \quad (14)$$

which is the semi-discrete matrix form. To proceed with the full discretisation, the theta method will be employed. Introduce therefore a grid of $n + 1$ points over the time domain $(0; T)$, each point identified by $t^{(n)}$. Consider for simplicity the following notations: $\underline{u}^{(n)} \equiv \underline{u}(t^{(n)})$ and $\underline{p}^{(n)} \equiv \underline{p}(t^{(n)})$. By applying the theta method to the semi discretised matrix form (14) with the previously mention time grid, the following set of equations can be obtained:

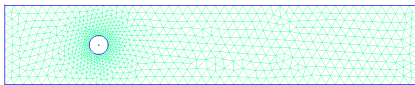
$$\begin{cases} M \frac{\underline{u}^{(n+1)} - \underline{u}^{(n)}}{\Delta t} + A \left[\theta \underline{u}^{(n+1)} + (1 - \theta) \underline{u}^{(n)} \right] + \theta C \left(\underline{u}^{(n+1)} \right) \underline{u}^{(n+1)} + (1 - \theta) C \left(\underline{u}^{(n)} \right) \underline{u}^{(n)} + \\ - B^T \left[\theta \underline{p}^{(n+1)} + (1 - \theta) \underline{p}^{(n)} \right] = \theta \underline{\mathcal{F}} \left(t^{(n+1)} \right) + (1 - \theta) \underline{\mathcal{F}} \left(t^{(n)} \right) \\ B \underline{u}^{(n+1)} = 0 \end{cases} \quad (15)$$

which are indeed a fully discretised form of the Navier Stokes equations.

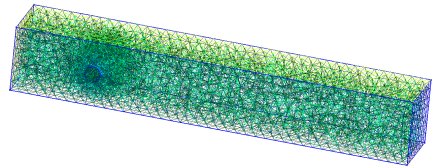
3. Mesh Generation and Definitions

The spatial discretization of the computational domain is a critical step in ensuring the accuracy and stability of the Navier-Stokes numerical solution. For this study, the mesh generation was performed using the open-source finite-element mesh generator **Gmsh**.

To effectively resolve the flow physics, particularly the steep velocity gradients within the boundary layers, we implemented a variable mesh density strategy.



(a) Gmsh representation of a 2d domain.



(b) Gmsh representation of a 3D domain.

Figure 1: Computational grids generated via **Gmsh**.

The domains were discretised using the Python API of the **Gmsh** library. This approach allowed for a fully parametric definition of the geometry. To ensure an accurate representation of the physics, the meshing process was dynamically controlled using distance parameters which automatically refined the local grid resolution in critical regions—such as the obstacles layers. In the case of Figure 1a, the mesh refines in proximity of the circle and gradually gets coarser as soon as the distance increases. A similar approach is shown in Figure 1b as well, where a solid cylindrical structure represents the obstacle where the mesh triangulations are finer. This script-based workflow facilitated the automated generation of diverse mesh configurations.

3.1. Test Cases

The following test cases serve as benchmark problems for the solver. Originally defined in 1996 by M. Schäfer and S. Turek [4], they have since become a de facto standard for assessing the performance of CFD solvers.

3.1.1 2D case

To evaluate the flow solver under different kinematic regimes, the previously cited paper proposes different numerical simulations to perform on the following 2D domain (Figure 2).

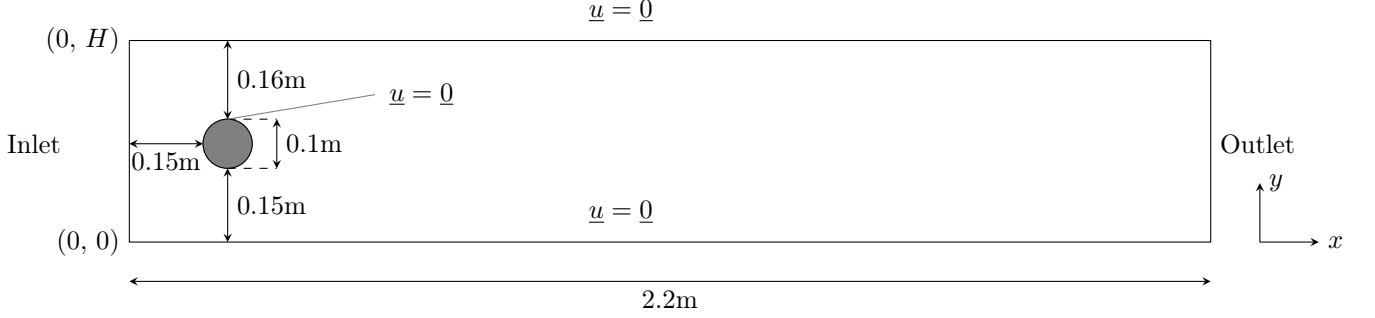


Figure 2: Geometry of 2D test cases with boundary conditions.

The fluid enters the channel through the inlet boundary with a prescribed velocity profile, while no-slip conditions ($u = v = 0$) are imposed on the top and bottom walls, as well as on the cylinder surface. Three distinct test cases were investigated, characterized by the mean inlet velocity $\bar{U}$:

- **Test Case 1 (Low-Re Steady):** A constant low mean velocity of $\bar{U} = 0.3$ m/s. This configuration corresponds to a laminar flow regime, expected to produce steady wake patterns.
- **Test Case 2 (High-Re Steady):** A constant increased mean velocity of $\bar{U} = 1.5$ m/s. In this regime, the Reynolds number is significantly higher, resulting in a more turbulent behaviour.
- **Test Case 3 (High-Re Pulsatile):** A time-dependent inflow where the mean velocity $\bar{U} = 1.5$ m/s is modulated by a sinusoidal factor. This introduces a transient forcing frequency to the system, defined as:

$$U_{in}(t) = 4 \cdot \bar{U} \cdot (H - y) \cdot y \cdot [\sin(\pi t/8)] / H^2 \quad (16)$$

where H represents the height of the domain and y the coordinate along the y-axis.

3.1.2 3D cases

The investigation was subsequently extended to a three-dimensional domain to capture span-wise flow features and three-dimensional instabilities. The geometry was generated taking inspiration from the M. Schäfer and S. Turek benchmark [4], both an obstacle with square section (Figure 3) and circular square section (Figure 4) have been employed.

Consistent with the 2D setup, no-slip conditions were applied to the cylinder and channel walls, while appropriate boundary conditions were imposed on the lateral surfaces to mimic an infinite span or confined channel. The simulation campaign consisted of three test cases:

- **Test Case 1 (Low-Re Steady):** A constant mean inflow velocity of $\bar{U} = 0.45$ m/s. This regime serves as a validation baseline, where the flow is expected to remain predominantly laminar and two-dimensional.
- **Test Case 2 (High-Re Steady):** A constant mean inflow velocity of $\bar{U} = 2.25$ m/s. At this higher Reynolds number, the flow transitions to a three-dimensional state.
- **Test Case 3 (High-Re Unsteady):** A time-varying inflow with a mean velocity of $\bar{U} = 2.25$ m/s, modulated by a sinusoidal profile. In this case a transient forcing frequency to the system was added, defined as:

$$U_{in}(t) = 16 \cdot \bar{U} \cdot (H - y) \cdot (W - z) \cdot y \cdot z \cdot [\sin(\pi t/8)] / H^4 \quad (17)$$

where H represents the height of the domain, W the width of the domain and (y, z) the coordinate along the y and z axis.

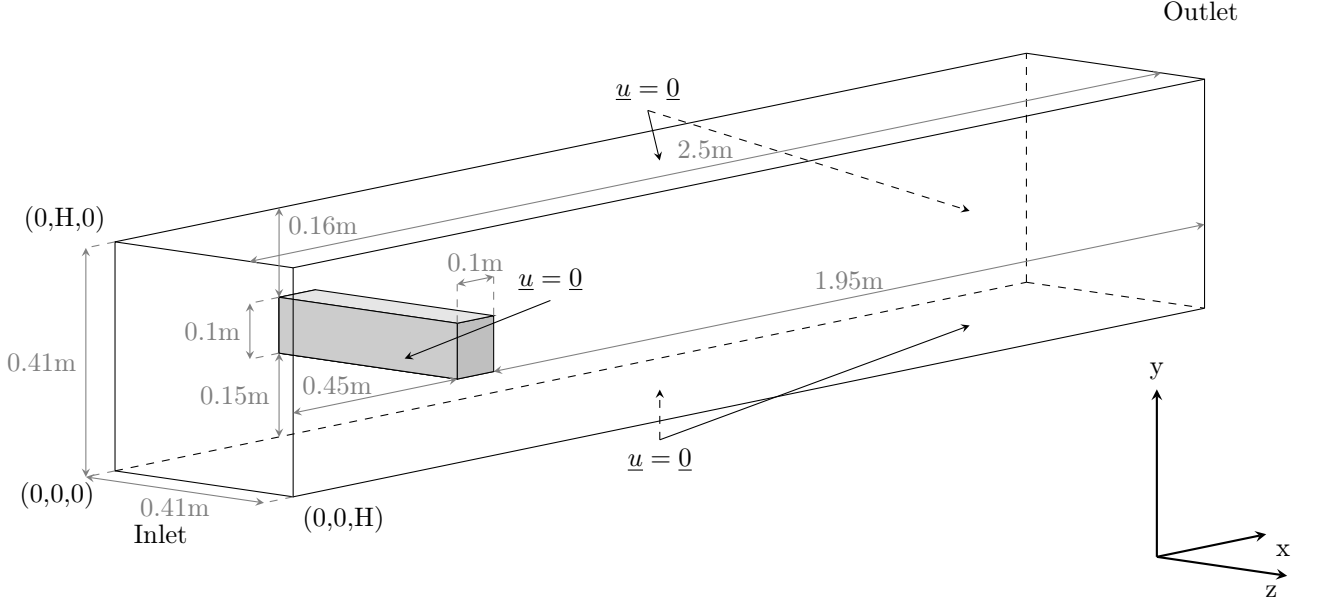


Figure 3: Geometry of 3D test cases with boundary conditions for flow around an obstacle with square section ("parallelepiped").

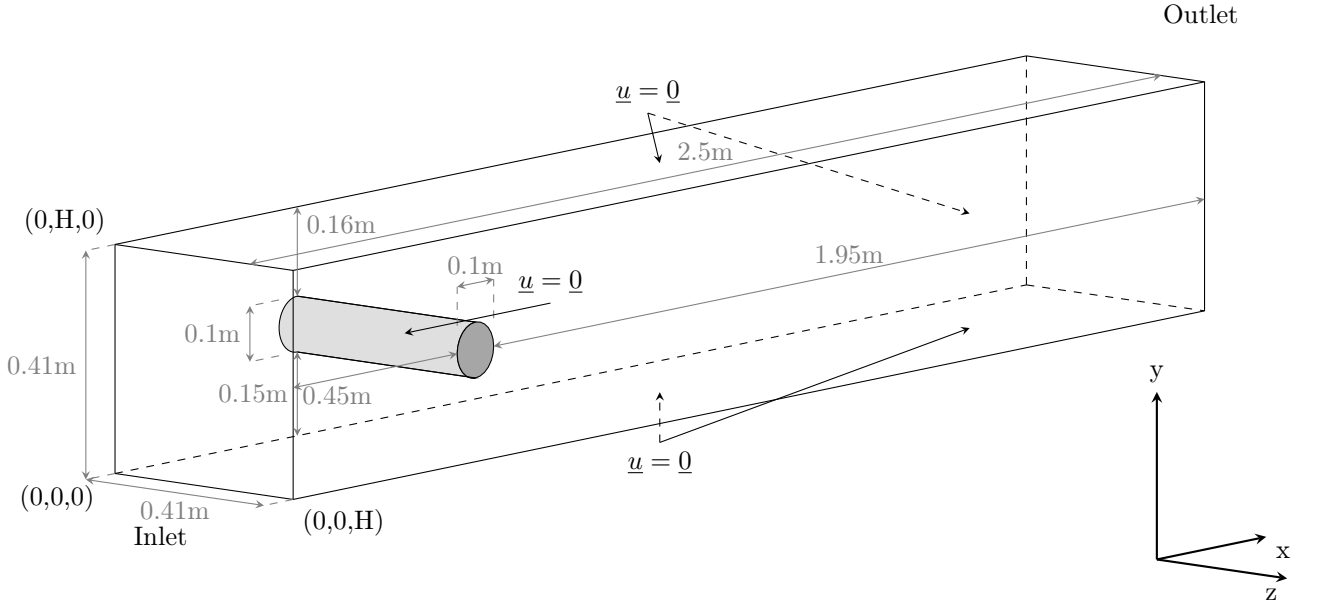


Figure 4: Geometry of 3D test cases with boundary conditions for flow around an obstacle with circular section ("cylinder").

3.2. Drag-Lift coefficients

The computation of drag-lift coefficients is required during the simulation of all test cases and defined as followed:

$$C_D = \frac{F_D}{\bar{U}^2 A} \quad \wedge \quad C_L = \frac{F_L}{\bar{U}^2 A}, \quad (18)$$

where F_D (parallel to the flow direction) and F_L (perpendicular to the the flow direction) are the corresponding forces applied on the object, $\bar{U}$ is the mean velocity and A is the characteristic area (D for **2D**, $D \cdot H$ for **3D**) In particular, those are non-dimensional quantities used to measure the aerodynamic force acting on the cylinder placed inside the channel.

Before diving into the definition of the forces, we need to introduce the definition of **Stress Tensor** σ whose formula is the following:

$$\sigma = -p\mathbf{I} + \mu(\nabla \underline{u} + (\nabla \underline{u})^T)$$

where p is the pressure, $\mathbf{I}$ is the identity matrix, μ (or $p\nu$) is the dynamic viscosity, $\nabla \underline{u} + (\nabla \underline{u})^T$ is the rate of strain tensor.

The total force vector $\underline{F}$ acting on the obstacle is the surface integral of the stress tensor dotted with the normal vector $\hat{\eta}$ (pointing out of the fluid and into the obstacle):

$$\underline{F} = \int_S \sigma \cdot \hat{\eta} dS$$

The Drag force (F_D) and Lift force (F_L) are defined as the components of the total force vector $\underline{F}$ parallel and perpendicular to the flow direction, respectively. Assuming the flow is aligned with the x-axis:

$$F_D = F_x \quad F_L = F_y \quad (19)$$

4. Description of the Solver

The computational framework is built upon the `deal.II` finite element library, which handles mesh management and Degrees of Freedom (*DoFs*) distribution. For the linear algebra, which in particular refers to the parallel distribution of the solver and the use of preconditioners, we utilized the `Trilinos` library. The solution procedure follows a nonlinear Newton iteration loop to linearize the convective term. The Jacobian systems obtained at each Newton step are solved using the GMRES algorithm. To verify the correctness of the implementation, we conducted convergence studies and flow analysis on three specific benchmarks, performing simulations for both 2D and 3D geometries. In addition to that, several preconditioners were used, in order to obtain a better conditioning in the linear systems' matrices.

4.1. Parameters Setting

In `main` function parameters are parsed to configure the problem setup. While the temporal parameters are fixed, the physical properties and geometry can be controlled via the command-line flags listed in Table 1.

Flag	Variable	Description
-f	<code>mesh_file_name</code>	Path to the Gmsh grid file (.msh)
-v	ν (viscosity)	Kinematic viscosity of the fluid
-u	U_{max}	Max inflow velocity
-theta	θ	Time-stepping parameter (suggested 0.5 or 1.0)
-d	d (dim)	Spatial dimension (2 or 3)
-T	T_{end}	Total simulation time
-dt	Δt	Time step size
-td	<code>time_dependency</code>	Allows to choose between inlet conditions (17)/(16) or ramped parabolic inlet velocity

Table 1: Command-line arguments available for simulation configuration.

If not specified, parameters follow their default initialization:

```
// default settings
std::string mesh_file_name = "../mesh/mesh3D_example.msh";
double viscosity = 1.0;
double theta = 1.0; // Backward Euler
double U_max = 0.45;
int dim = 3;
double T = 2.;
double delta_t = 0.02;
```

Listing 1: Configuration of simulation parameters.

Depending on the provided dimension flag, the code instantiates the solver for either 2D or 3D domains and initiates the `run_time_simulation` routine.

4.2. System Initialization

The initialization phase is handled by the `initialize_system` function, which first constructs the parallel grid by reading a Gmsh format file and partitioning the `Triangulation` across the available MPI processes.

The finite element space is built using `FE_SimplexP` basis functions, combining the velocity and pressure fields into a single coupled `FESystem`. To ensure the numerical stability of the system arising from the incompressibility constraint, the finite element spaces were chosen to satisfy the Ladyzhenskaya-Babuška-Brezzi (LBB) inf-sup condition 13. We adopted the stable Taylor-Hood element pair ($\mathbb{P}^2/\mathbb{P}^1$). This choice suppresses spurious pressure modes and guarantees optimal convergence rates for both variables.

The block sparsity patterns are generated by defining the specific coupling between velocity and pressure degrees of freedom. Finally, all distributed vectors including the solution, Newton update, and right hand side are reinitialized using `TrilinosWrappers` based on the locally owned degrees of freedom.

4.3. System Assemble

The assembly of the linear system is performed within the `assemble` function, which iterates over all locally owned cells to construct both the Jacobian matrix and the residual vector. For each cell, we initialize `FEValues` objects to evaluate the shape functions, gradients, and quadrature weights. The implementation distinguishes between the system matrix assembly (controlled by the `assemble_matrix` flag) and the right-hand side.

During the matrix assembly, we construct the Jacobian of the non-linear residual. This includes the mass matrix contribution from the time derivative ($\frac{1}{\Delta t}$), the viscous diffusion term ($\theta\nu(\nabla\mathbf{u}, \nabla\mathbf{v})$), and the linearized convective term. An auxiliary `cell_pressure_mass_matrix` is assembled for the following initialization of a block preconditioner.

The RHS vector (`local_rhs`) accumulates the residual of the weak formulation. The time integration follows a θ scheme. Boundary conditions are handled by identifying faces on the outlet boundary (ID 2); here, a traction term $-p_{out}\hat{\eta} \cdot \mathbf{v}$ is integrated using `FEFaceValues`.

Finally, the local contributions are distributed to the global system using `AffineConstraints`. A critical distinction is made here: if `initial_step` is true, we apply inhomogeneous boundary constraints (`nonzero_constraints`); otherwise, for Newton updates, we apply homogeneous constraints (`zero_constraints`) to solving for the increment $\delta\mathbf{u}$.

4.4. Newton Iteration

As already stated in the introduction of the section, the non-linear problem is solved using a Newton-Raphson iterative scheme. At each step k , we calculate the Newton update $\delta\mathbf{u}$ by solving the linearized system:

$$J(\mathbf{u}_k)\delta\mathbf{u} = -\underline{R}(\mathbf{u}_k)$$

where J is the Jacobian matrix and $\underline{R}$ is the residual vector. To ensure global convergence, specifically when the initial guess is far from the solution, we employ a backtracking line search strategy. If a Newton step fails to reduce the residual enough below the tolerance, the damping factor α is halved at each iteration and the update is computed again:

$$\alpha \leftarrow 0.5\alpha$$

This process repeats until the monotonicity condition is satisfied:

$$\|\underline{R}(\mathbf{u}_k + \alpha\delta\mathbf{u})\| < \|\underline{R}(\mathbf{u}_k)\|$$

Within each Newton step, the linear system is inverted using the Flexible Generalized Minimal Residual method (FGMRES), provided by the `Trilinos` framework.

4.5. Drag and Lift Coefficients

The computation of the aerodynamic coefficients is performed within the `compute_lift_drag` function. This procedure is executed as a post-processing step at the end of each time step, once the Newton solver has successfully converged.

The solver computes the total force vector $\underline{F}$ acting on the cylinder by integrating the stress tensor σ over the cylinder's surface S . Since the code can't computationally solve the integral in a continuous domain, we used the **Gaussian Quadrature**, which transforms the integral into a finite summation over discrete quadrature points, to approximate it.

The implementation utilizes the `FEFaceValues` class from the `deal.II` library to evaluate shape functions and gradients on the boundary.

The algorithm proceeds through the following steps:

1. **Boundary Iteration:** The solver iterates through all locally owned active cells `dof_handler.active_cell_iterators()` to identify faces located on the cylinder boundary.
2. **Stress Reconstruction:** At each quadrature point, the local pressure `p_val` and velocity gradients `grad_u` are extracted. These are used to reconstruct the viscous stress tensor `stress`.
3. **Force Accumulation:** The stress tensor is contracted with the outward normal vector `n` to obtain the local force density. This density is multiplied by the `JxW` term and accumulated into the `total_force` vector.

Once the accumulation of forces is complete, the code sums the forces across all MPI processes and stores the final result in the local vector `global_force`.

Finally, the total force is normalized by the square of the reference velocity and the reference length to derive the drag (C_D) and lift (C_L) coefficients, which are computed from the accumulated `global_force[0]` and `global_force[1]`, respectively.

4.6. Preconditioners

To enhance performance, the solver employs preconditioning techniques to speed up convergence, which leads significant improvements in terms of iterations, and it avoids convergence failures during the simulation. Since the linear system arising from the Navier-Stokes equations is of the saddle-point type, standard black-box preconditioners often fail; therefore, we implement physics-based block preconditioners that exploit the 2×2 block structure of the system matrix:

$$\mathcal{A} = \begin{pmatrix} A & B^T \\ B & 0 \end{pmatrix} \quad (20)$$

where A represents the velocity stiffness matrix (including time, convective, and viscous terms), and B and B^T represent the divergence and gradient operators, respectively.

In the following

4.6.1 Block Diagonal Preconditioner

The simplest approach implemented is the Block Diagonal Preconditioner (class `BlockPreconditioner`), which doesn't consider the coupling between velocity and pressure fields. The preconditioning matrix $\mathcal{P}_{diag}$ is defined as:

$$\mathcal{P}_{diag}^{-1} = \begin{pmatrix} \tilde{A}^{-1} & 0 \\ 0 & \tilde{M}_p^{-1} \end{pmatrix} \quad (21)$$

In our implementation, the inverses $\tilde{A}^{-1}$ and $\tilde{M}_p^{-1}$ are approximated by solving the respective blocks using the **Conjugate Gradient** (CG) method preconditioned with an **Incomplete LU** (ILU) factorization. Despite of its simple implementation, this preconditioner is generally less robust for non-stationary flows with significant time-dependency.

4.6.2 Block Triangular Preconditioner

The Block Triangular Preconditioner (class `PreconditionBlockTriangular`) improves upon the diagonal approach by accounting for the one-way coupling from velocity to pressure. It approximates the system using a lower triangular form:

$$\mathcal{P}_{tri} = \begin{pmatrix} A & 0 \\ B & M_p \end{pmatrix} \quad (22)$$

The application of this preconditioner involves a two-step process:

1. **Velocity Solve:** Solve $A\mathbf{u} = \mathbf{f}$ using **GMRES** with **ILU preconditioning**.
2. **Pressure Solve:** Update the pressure residual $\mathbf{r}_p = \mathbf{g} - B\mathbf{u}$ and solve $M_p \mathbf{p} = \mathbf{r}_p$ using **CG** with **ILU preconditioning**.

4.6.3 Schur Complement Preconditioner

The most strategical preconditioner employed is the Block Schur Preconditioner (class `BlockSchurPreconditioner`), which is the currently active method in the solver. This approach utilizes the

Upper Triangular factorization of the block system:

$$\mathcal{P}_{Schur}^{-1} = \begin{pmatrix} A & B^T \\ 0 & S \end{pmatrix}^{-1} \quad (23)$$

where $S = -BA^{-1}B^T$ is the Schur complement. Since calculating S explicitly is computationally expensive, we approximate it using the pressure mass matrix scaled by the viscosity ν and the time-stepping coefficient γ (where $\gamma \propto 1/\Delta t$). The approximation used is:

$$\tilde{S}^{-1} \approx -(\nu + \gamma)M_p^{-1} \quad (24)$$

The application of this preconditioner proceeds as follows:

1. **Pressure Predictor:** Solve the approximated Schur system for pressure: $M_p \underline{p} = -(\nu + \gamma)^{-1} \underline{g}$. This is performed using **FGMRES** on the pressure mass matrix.
2. **Velocity Update:** Compute the velocity residual $\underline{r}_u = \underline{f} - B^T \underline{p}$ and solve $A \underline{u} = \underline{r}_u$ using **ILU factorization**.

This method effectively handles the saddle-point nature of the problem, providing the most stability for the Reynolds numbers considered in this study.

4.7. Output

The visualization of results is managed by a function which uses the **DataOut** class to export the finite element solution of each time instant. The solution vector is decomposed into its physical components, with velocity interpreted as a vector field of dimension dim and pressure as a scalar. Before saving, the Reynolds number is computed based on the domain dimensionality and the inlet mean velocity U_{mean} . This value is used to generate dynamic filenames (e.g., `100Re-NS_Solution_...`).

The simulation results were visualized and analyzed using the open-source software **ParaView**. Since the output data was stored in the parallel VTK format (`.pvtu`), **ParaView** automatically reconstructed the global computational domain by aggregating the data fragments from individual MPI processes. The time-dependent evolution of the flow was visualized by loading the sequence of solution files which represented each a single time step.

5. The Splitting Method of Chorin and Temam

The Chorin-Temam method is a fractional-step scheme used to decouple the velocity and pressure fields in the incompressible Navier-Stokes equations.

5.1. Algorithm

Given the velocity $\underline{u}^{(n)}$ at time $t^{(n)}$, we find $\underline{u}^{(n+1)}$ and $p^{(n+1)}$ through the following steps:

1. **Prediction Step:** Solve for the intermediate velocity field $\tilde{\underline{u}}^{(n+1)}$ by considering convection and diffusion.

$$\begin{cases} \frac{\tilde{\underline{u}}^{(n+1)} - \underline{u}^{(n)}}{\Delta t} - \nu \Delta \tilde{\underline{u}}^{(n+1)} + (\underline{u}^* \cdot \nabla) \underline{u}^{**} = \underline{f}^{(n+1)} & \text{in } \Omega, \\ \tilde{\underline{u}}^{(n+1)} = \underline{0} & \text{on } \partial\Omega; \end{cases} \quad (25)$$

2. **Pressure Poisson Equation:** Solve for the pressure p^{n+1} that will enforce the incompressibility constraint.

$$\begin{cases} \frac{\underline{u}^{(n+1)} - \tilde{\underline{u}}^{(n+1)}}{\Delta t} + \nabla p^{(n+1)} = \underline{0} & \text{in } \Omega, \\ \text{div}(\underline{u}^{(n+1)}) = 0 & \text{in } \Omega, \\ \underline{u}^{(n+1)} \cdot \hat{n} = 0 & \text{on } \partial\Omega, \end{cases} \quad (26)$$

3. **Projection Step:** Update the velocity field to the next time level using the pressure gradient.

$$\underline{u}^{(n+1)} = \underline{u}^* - \Delta t \nabla p^{(n+1)} \quad (27)$$

5.2. Convergence

The Chorin-Temam projection method is widely adopted in computational fluid dynamics (CFD) due to its efficiency in decoupling the velocity and pressure fields. Its behaviour and benefits are briefly described below. For the classical scheme described above (25), the following convergence rates are typically observed:

- **Velocity:** The method achieves first-order accuracy in time, $\mathcal{O}(\Delta t)$, in the $[L^2(\Omega)]^d$ norm.
- **Pressure:** The pressure field generally converges at a rate of $\mathcal{O}(\Delta t^{1/2})$ in the $L^2(\Omega)$ norm, although this can be improved to $\mathcal{O}(\Delta t)$ with incremental variants.
- **Convergence:** Depends by the choice of finite element spaces $(\mathbb{P}^k/\mathbb{P}^{k-1})$, typically $\mathcal{O}(h^{k+1})$ for velocity.

Key Points of the Algorithm

- **Operator Decoupling:** By splitting the momentum and continuity equations, the algorithm avoids solving a large system. Instead, it solves a sequence of simpler elliptic and parabolic problems.
- **Computational Efficiency:** The most expensive part is solving the Pressure Poisson Equation. Since the Laplace operator is symmetric and positive definite, highly efficient solvers like Conjugate Gradient (CG) can be used.
- **Linearization:** The semi-implicit approach $(\underline{u}^{(n)} \cdot \nabla) \underline{u}^*$ eliminates the need for the expensive Newton methods for the non-linear convective term within each time step, which was utilized in the classical solver.

6. Results and discussion

6.1. 2D test cases

6.1.1 2D Test Case 1

2D Test Case 1 represents a low $\mathbb{Re}$ ($\mathbb{Re} = 20$) simulation in the steady regime. The simulation has been carried out from $t_0 = 0s$ to $T = 5s$ with a discretization step $\Delta t = 0.02s$.

Figure 5 shows the temporal evolution of the drag and lift coefficients. We can identify two main phases in the flow development:

1. **Transient Phase:** At the beginning ($t < 1.5s$), the flow exhibits a startup transient. The drag coefficient shows a sharp initial overshoot (peak of approximately 7.3) on the fine mesh. The lift coefficient exhibits a distinct oscillation before convergence.
2. **Steady Phase:** After the previous phase ($t \geq 1.5s$), the flow converges into a stable state. Both drag and lift coefficients converge to their corresponding constant values ($C_D = 2.77$, $C_L = 2.77$). While the drag coefficient converges monotonically, the lift has small oscillations that decrease over time.

Since both coefficients converge, the numerical solution has reached a steady state. Finally, the high drag implies that the object is not aerodynamic and the lift, which is very close to zero, indicates that the flow pushes equally on the top and bottom; thus, the object (obstacle) isn't generating any force to fly up or down. Figure 12 shows a series of snapshots of the simulation once the steady regime has been reached ($t = 5$).

6.1.2 2D Test Case 2

2D Test Case 2 represents a higher $\mathbb{Re}$ ($\mathbb{Re} = 100$) simulation in the unsteady regime. The simulation was performed over a time interval of $T = 5s$. The aerodynamic coefficient results are presented in Figure 6. (for both coarse and fine meshes).

The numerical solution in this specific setup maintains the symmetry of the flow, converging to a steady state rather than a periodic one.

Observation:

- **Drag Coefficient (C_D):** At the beginning ($t < 0.5s$), a sharp initial overshoot occurs, reaching a peak value of approximately 1.55 on the fine mesh. Following this transient spike, the drag converges very quickly. Note: we can observe a small difference among the results of the fine and coarse meshes ($C_{D,fine} \approx 1.33$ and $C_{D,coarse} \approx 1.45$) implying the necessity of better spatial resolution to correctly capture the boundary layer gradients and the resulting drag forces at $\mathbb{Re} = 100$.
- **Lift Coefficient (C_L):** The behaviour is a damped oscillation where for each period the amplitude decreases over time. On the fine mesh, the lift amplitude decays rapidly, settling to a constant value very close to zero ($C_L \approx -0.005$), referring a symmetric flow field. The coarse mesh shows a slightly large deviation from zero ($C_L \approx -0.019$) and slower damping, further suggesting that the coarser discretization

introduces slight numerical asymmetries or has higher numerical dissipation.

Figure 13 shows a series of snapshots of the simulation once the steady regime has been reached ($t = 5$).

6.1.3 2D Test Case 3

2D Test Case 3 represents a high Re ($Re = 100$) simulation in the unsteady regime. The simulation has been carried from time $t_0 = 0s$ to $T = 8s$ with discretisation step $\Delta t = 0.02s$. The drag and lift coefficients plots in Figure 7 show that drag follows the temporal evolution of the imposed inlet velocity, reaching a maximum at mid-time ($t = 4s$) before decreasing symmetrically, while the lack of oscillation indicates that the flow remains in a laminar regime. Lift on the other hand remains small compared to drag, indicating almost symmetric flow, with minor unsteady deviations. Comparison between fine and coarse meshes shows good convergence for drag, while lift exhibits higher mesh sensitivity.

Figures 14 and 15 shows a selection of snapshots from the solution at different time steps, both on the fine and the coarse mesh.

6.2. 3D test cases

6.2.1 3D Test Case 1

3D Test Case 1 represents a low Re ($Re = 20$) simulation in the steady regime. The simulation has been carried out from $t_0 = 0s$ to $T = 4s$ with discretisation step $\Delta t = 0.02s$, with an initial ramping parabolic inlet velocity with ramp duration $\tilde{t} = 0.25s$ and average velocity $|\underline{u}_{mean}| = 0.2 \frac{m}{s}$.

Figure 8 shows the evolution of the drag and lift coefficients in time. With respect to the drag coefficient C_D , a characteristic feature of the initial phase of all simulations is a sharp transient spike occurring within the first 0.25s, which is coherent with the ramped inlet velocity, where the fluid accelerates from rest to maximum velocity. In the square section obstacle this peak is much more pronounced, reaching a value of approximately 10.3, compared to 7.2 of the circular section. The same trend can be seen in the steady regime, where C_D settles around the value of 3.0 for the cylinder obstacle (in accordance to [4], adjusted for the coefficient definition (18)), while approximately 3.9 for the parallelepiped obstacle, due to the sharper edges of the square section, which cause a greater separation of the flow. The lift coefficient displays higher mesh sensitivity, where the fine mesh over the cylinder obstacle correctly predicts a small negative lift due to the asymmetry of the domain, while the coarser mesh fails to capture this feature, thus exhibiting a slightly positive lift. Interestingly, the fine mesh over the parallelepiped domain reaches a closer to zero result, probably as a result of the sharp corner separation which mitigates the effects of the asymmetric domain.

Figure 16 shows a series of snapshots of the simulation once the steady regime has been reached ($t = 4$). Comparison between 16b and 16f shows that the parallelepiped wake is wider and slower compared to the cylinder wake, thus explaining the higher drag coefficient. Plots 16d and 16h display the typical high-pressure stagnation on the front face of the obstacle.

6.2.2 3D Test Case 2

3D Test Case 2 represents an high Re ($Re = 100$) simulation in the steady regime. The simulation has been carried out from $t_0 = 0s$ to $T = 4s$ with discretisation step $\Delta t = 0.02s$, with an initial ramping parabolic inlet velocity with ramp duration $\tilde{t} = 0.25s$ and maximum velocity $|\underline{u}_{max}| = 2.25 \frac{m}{s}$. Looking at the graphs in Figure 9 we can clearly recognize two different phases:

1. **Transient Phase:** The initial large oscillations in both drag and lift represent the start-up phase of the simulation. Here the inlet velocity varies until it reaches the $\underline{u}_{max}$.
2. **Steady Phase:** After the initial transient, both coefficients settle into constant values.

Drag coefficient being positive means there is a constant resistive force, in particular when the flow becomes steady. On the other hand a lift that is close to zero indicates the flow is being carried out symmetrically. The difference in final values between the two plots (coarse vs. fine grid) shows clearly how the different mesh refinement creates variable results.

Comparing the results between the cylinder and parallelepiped mesh it is clearly visible how the drag coefficient for the rectangular parallelepiped stabilizes at a much higher value (between 2.1 and 2.3) compared to the rounded cylinder one. This is exactly the expected behaviour since a rectangular shape has more sharp corners that force flow separation. Just like the cylinder cases, the lift coefficient for the parallelepiped quickly decays after the startup phase ($t < 1.0s$) and flattens out to a tiny constant value (around 0.004).

6.2.3 3D Test Case 3

3D Test Case 3 represents an high Re ($Re = 100$) simulation in the unsteady regime. The simulation has been carried out from $t_0 = 0s$ to $T = 4s$ with discretisation step $\Delta t = 0.02s$, with the inlet velocity defined at (17) with maximum velocity $|\underline{u}_{max}| = 2.25$. As it is clearly visible in Figure 10, the unsteady test behaviour is completely different from the previous results obtained. In these tests, the Drag Coefficient (C_D) for all geometries starts near zero and smoothly, continuously rises over the entire 4 seconds duration. As regards the drag graphs, both the cylinder and parallelepiped version show a similar pattern, with the cylinder test having, as expected, a smaller value given the roundness of the geometry. On the other hand the parallelepiped drag reaches value between 2.1 and 2.3. The lift graphs shows surprisingly a different result for both the cylinder and parallelepiped version. For the cylinder, C_L forms a long, positive arch, peaking around $t=3.2s$ to $3.7s$ before beginning to fall. An important remark comes from the comparison between fine and coarse meshes, as generally coarse meshes tend to react sooner than fine meshes, probably given an higher uncertainty on data. For the parallelepiped, C_L continuously drops into negative values over the 4 seconds. While the lift coefficient curves exhibit different shapes across the tested geometries and mesh resolutions, these variations are fundamentally negligible. The slight positive or negative drift observed in the graphs is a standard numerical artifact. It typically arises from minor asymmetries in the mesh discretization (which indeed is visible in both the 2D 2 and 3D 3 meshes).

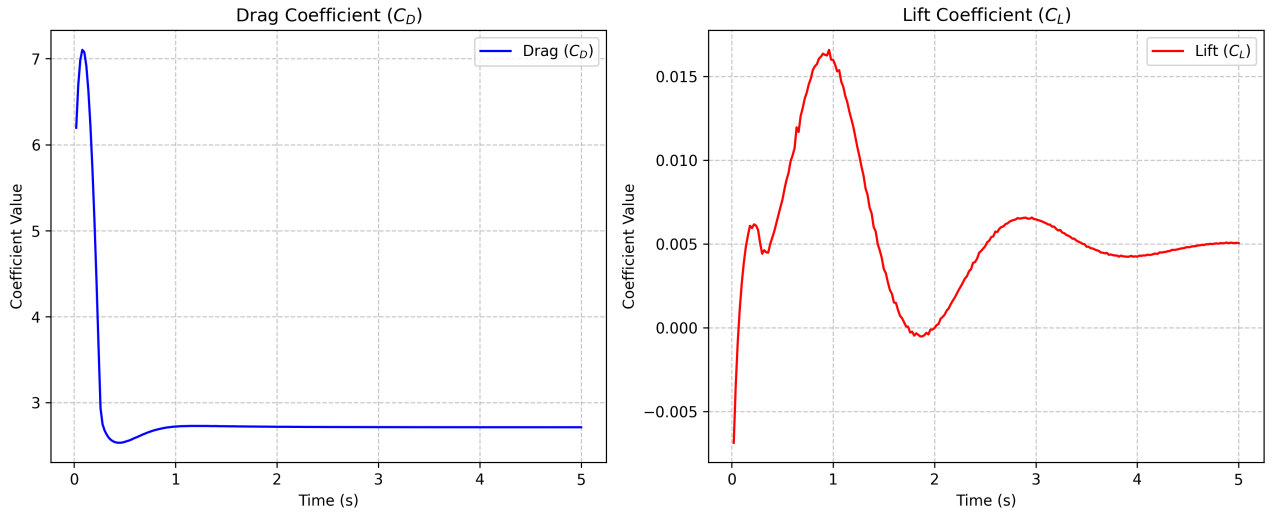
6.3. Chorin - Temam test case

To prove the superiority of the Chorin - Temam solver with respect to the fully coupled monolithic solver, a single simulation at $Re = 1000$ has been carried out. The simulation has been carried out from $t_0 = 0s$ to $T = 5s$ with discretisation step $\Delta t = 0.001s$, with an initial ramping parabolic inlet velocity with ramp duration $\tilde{t} = 0.25s$ and maximum velocity $|\underline{u}_{max}| = 15.0 \frac{m}{s}$. Figure 11 shows the evolution of the drag and lift coefficients in time. Drag coefficients shows an initial spike due to the ramped inlet, after which it settle around a mean value of circa 1.43, with periodic oscillations due to vortex shedding typical of such high Reynolds number. The same oscillatory regime can be observe in the lift coefficient as well, averaging around 0 (due to the almost symmetric nature of the 2D domain (2)) and with oscillation amplitude of around 0.38. The snapshots of the simulation at time $t = 5s$ in Figure 19 indeed show the most prominent characteristics of a fluid in such regime: fully developed von Kármán vortex street and roll-up of vortices in 19a and an high pressure stagnation at the front of the cylinder in 19b, with a strong low pressure zone (blue and purple) around the developing zone of the wake.

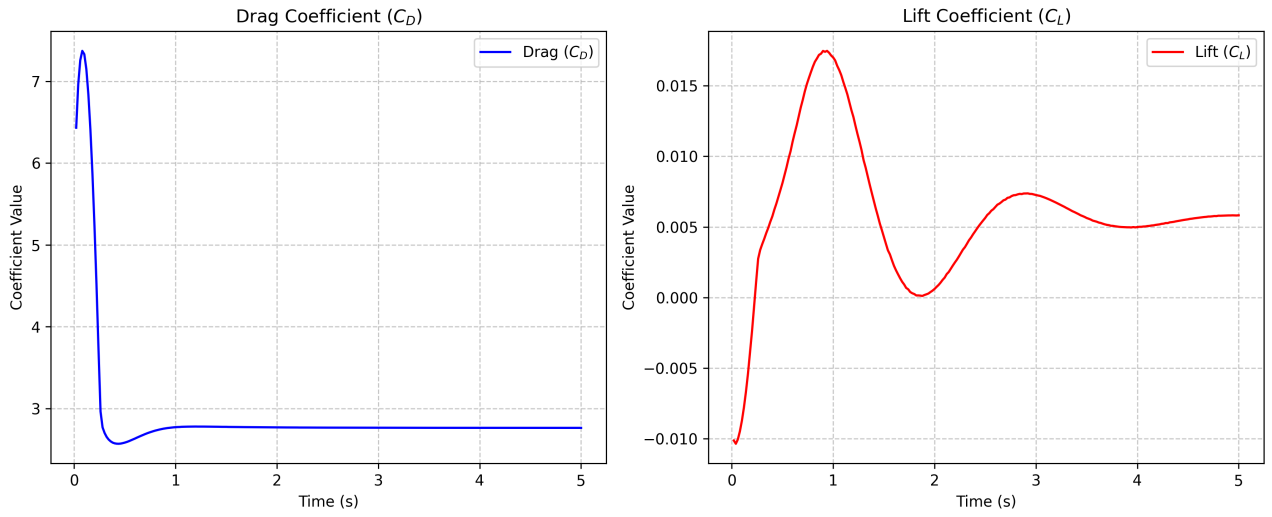
References

- [1] Franco Brezzi and Michel Fortin. *Mixed and Hybrid Finite Element Methods*, volume 15 of *Springer Series in Computational Mathematics*. Springer-Verlag, New York, NY, 1991.
- [2] Howard C. Elman, David J. Silvester, and A. J. Wathen. *Finite Elements and Fast Iterative Solvers: with Applications in Incompressible Fluid Dynamics*. Oxford University Press, second edition, 2014.
- [3] Alfio. Quarteroni. *Numerical Models for Differential Problems*. MS&A, Modeling, Simulation and Applications, 16. Springer International Publishing, Cham, 3rd ed. 2017. edition, 2017.
- [4] M. Schäfer, S. Turek, F. Durst, E. Krause, and R. Rannacher. *Benchmark Computations of Laminar Flow Around a Cylinder*, pages 547–566. Vieweg+Teubner Verlag, Wiesbaden, 1996.
- [5] L. Zhao and T. Heister. The deal.ii tutorial step-57: The incompressible, stationary navier stokes equations, 2017.

A. Drag and Lift Coefficients Plots

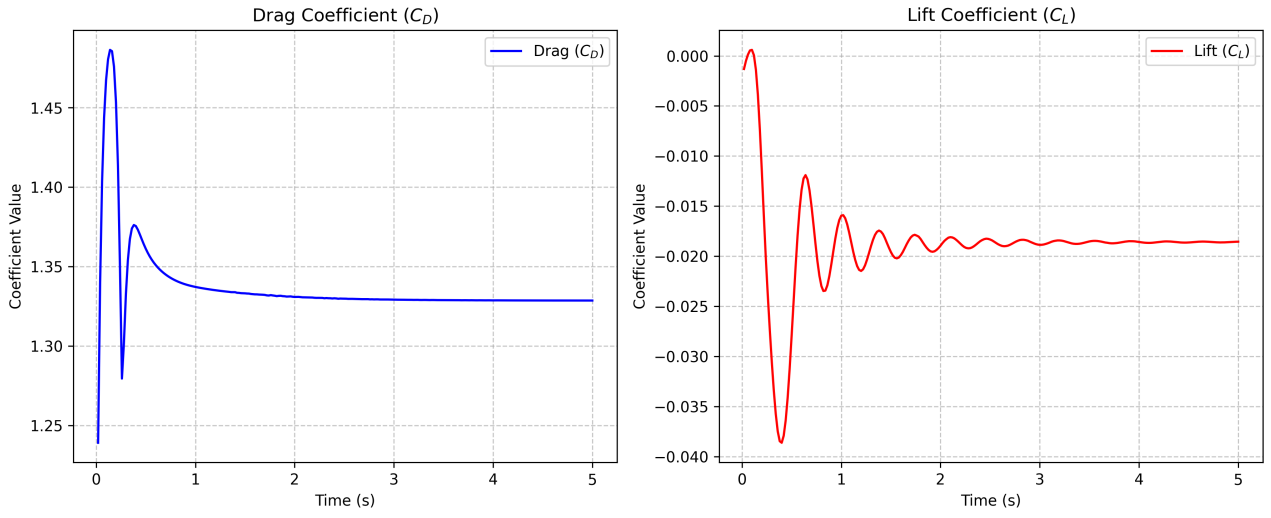


(a) Coarse mesh.

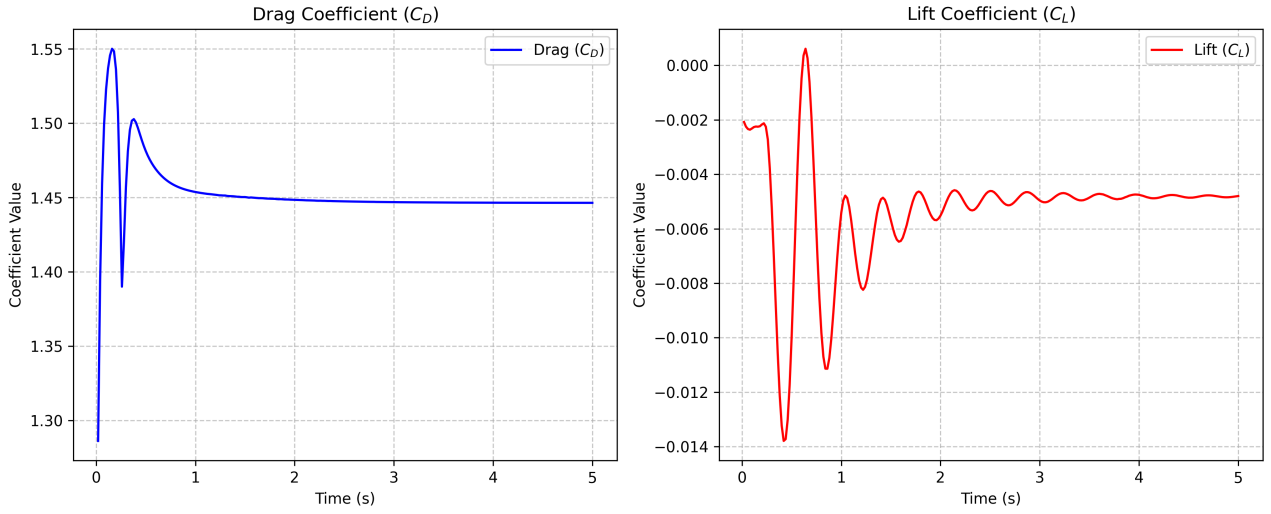


(b) Fine mesh.

Figure 5: Drag and lift coefficients for the 2D-1 steady test case.

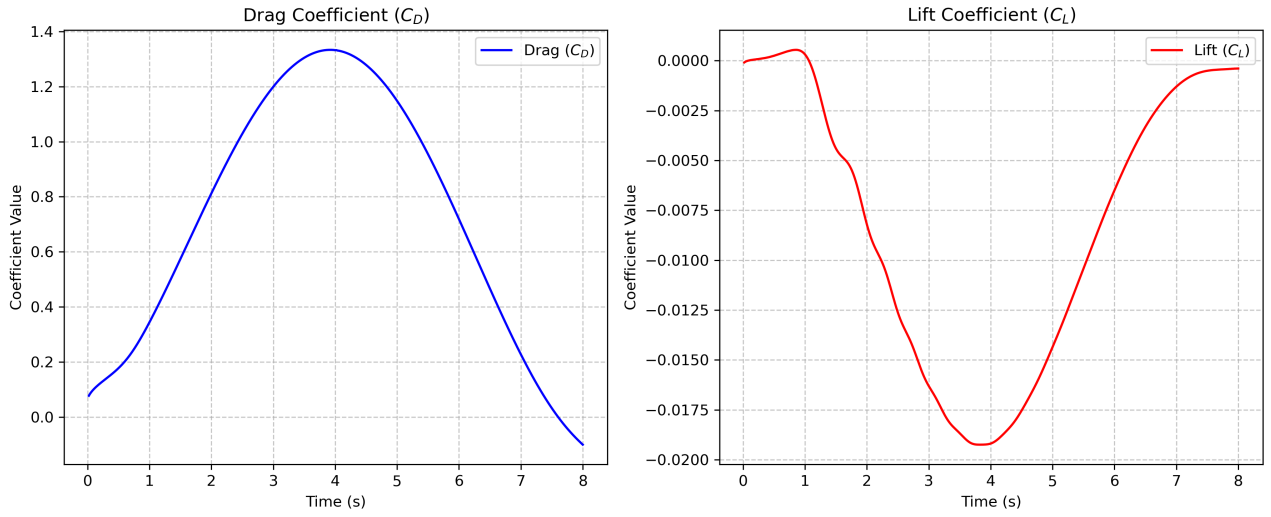


(a) Coarse mesh.

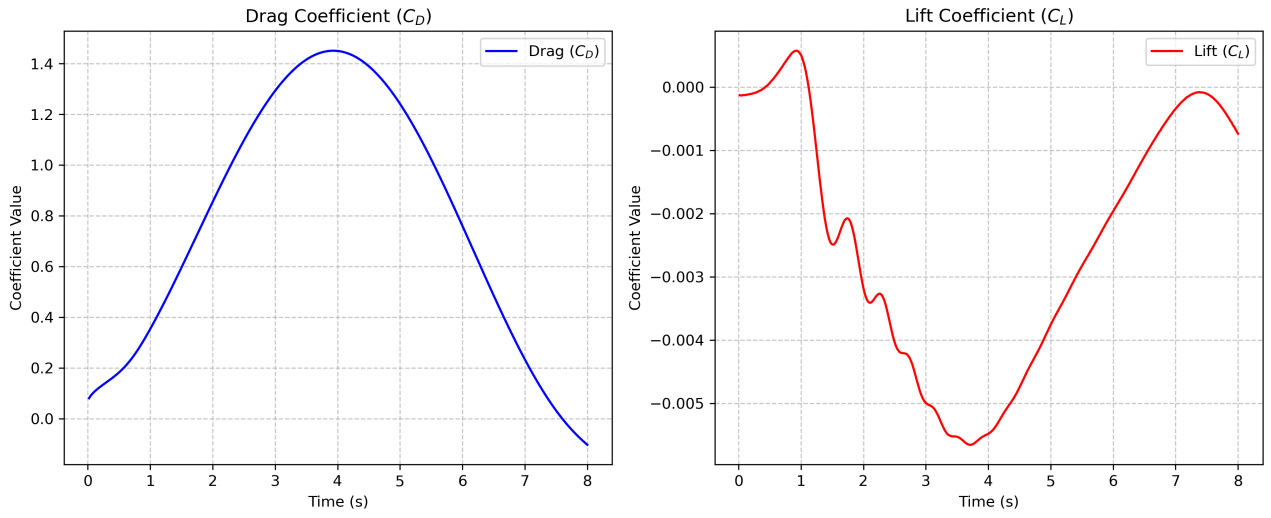


(b) Fine mesh.

Figure 6: Drag and lift coefficients for the 2D-2 steady test case.

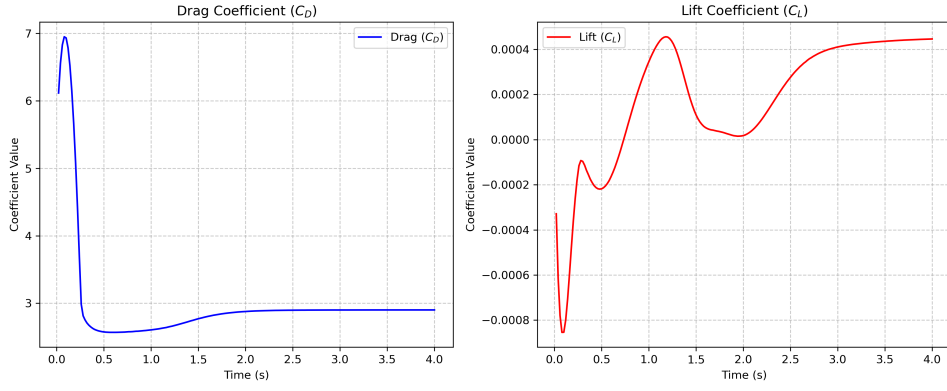


(a) Coarse mesh.

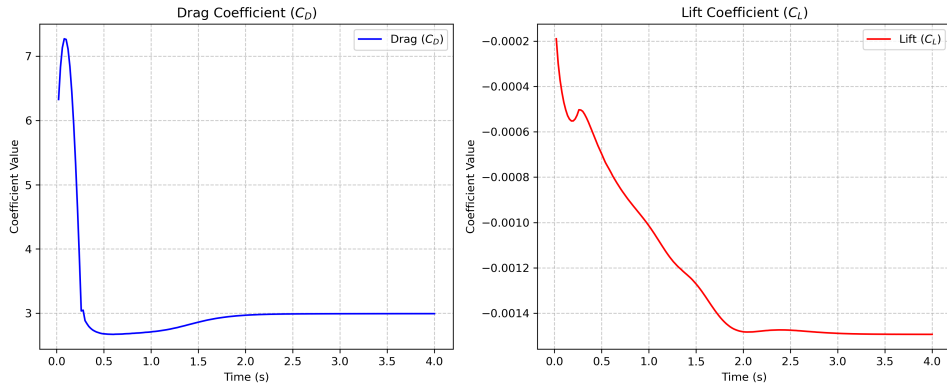


(b) Fine mesh.

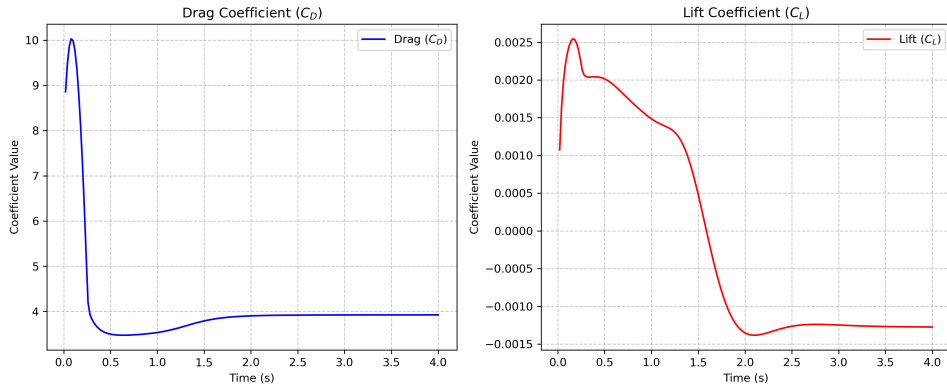
Figure 7: Drag and lift coefficients for the 2D-3 unsteady test case.



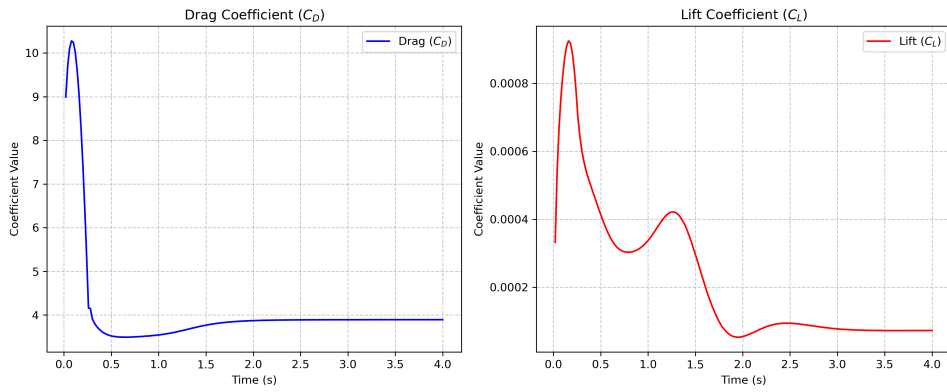
(a) Cylinder: Coarse mesh



(b) Cylinder: Fine mesh

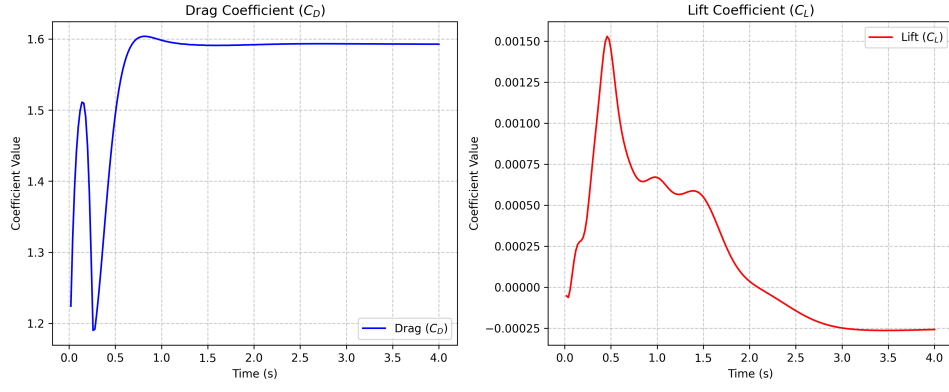


(c) Parallelepiped: Coarse mesh

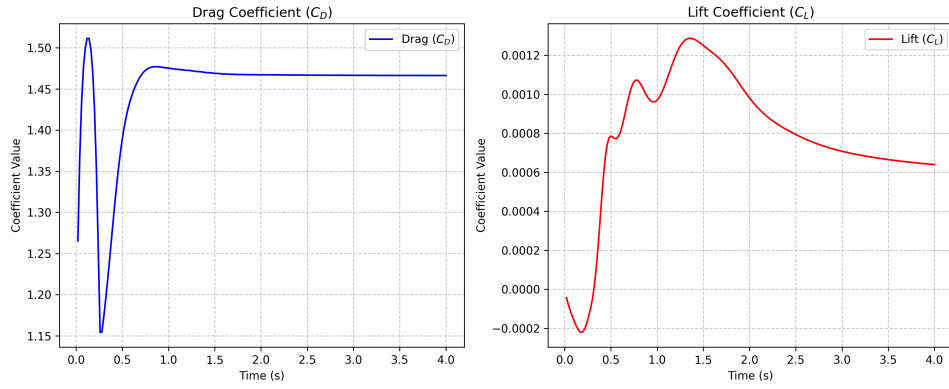


(d) Parallelepiped: Fine mesh

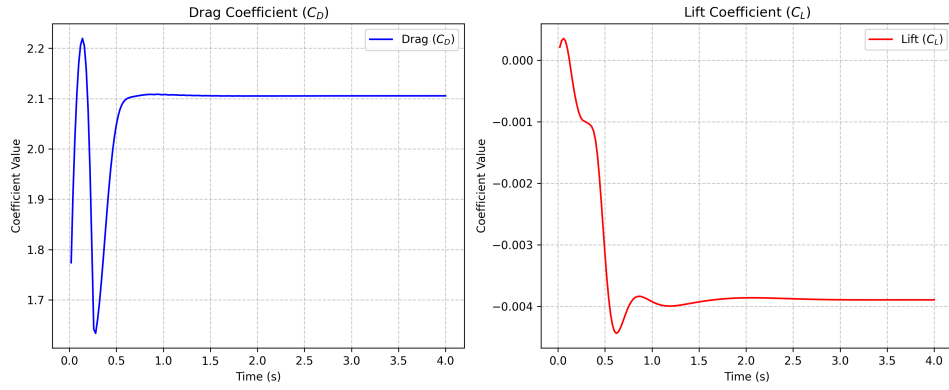
Figure 8: Drag and Lift coefficients (Cylinder and Parallelepiped) for the 3D-1 steady test case.



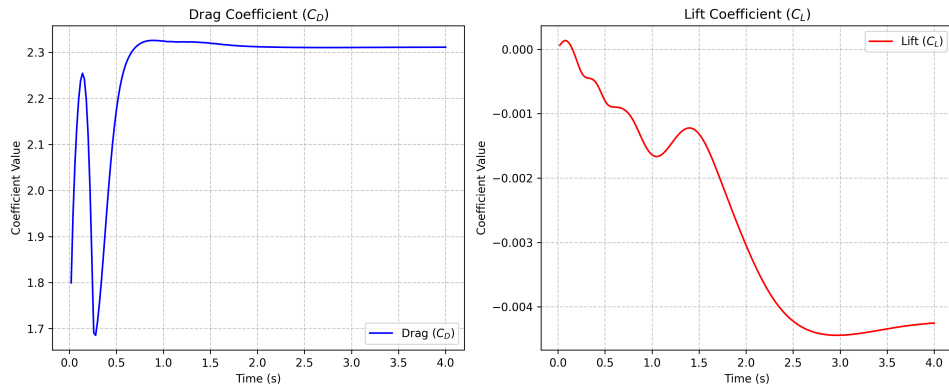
(a) Cylinder: Coarse mesh



(b) Cylinder: Fine mesh

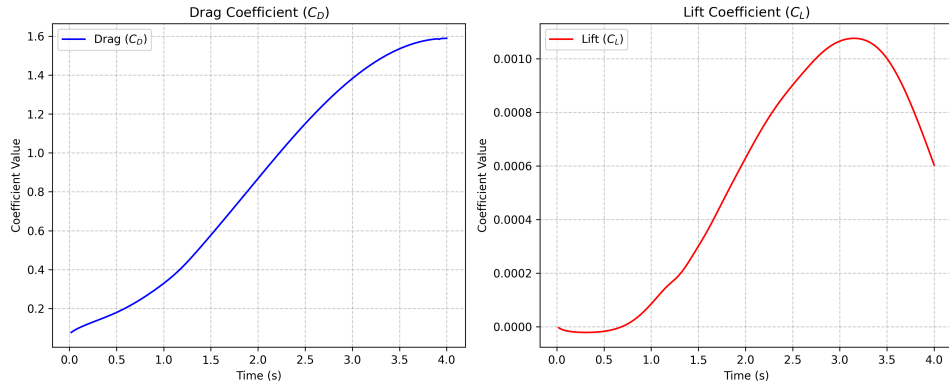


(c) Parallelepiped: Coarse mesh

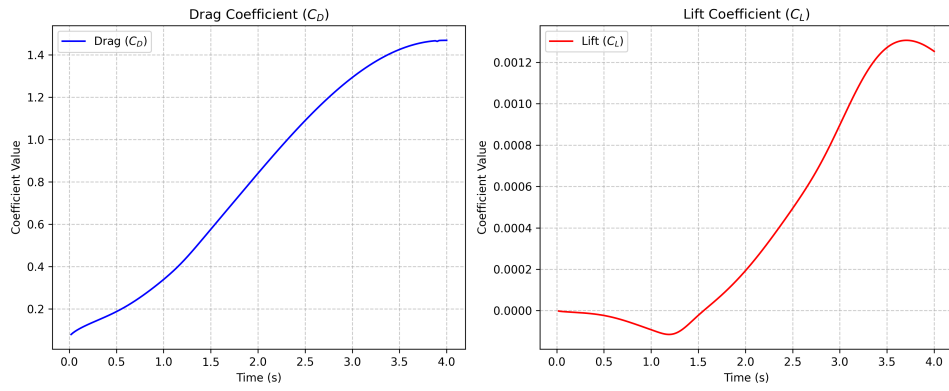


(d) Parallelepiped: Fine mesh

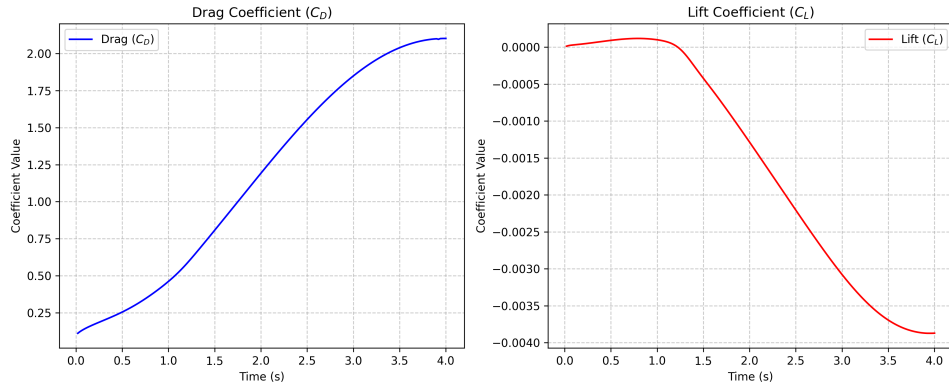
Figure 9: Drag and Lift coefficients (Cylinder and Parallelepiped) for the 3D-2 steady test case.



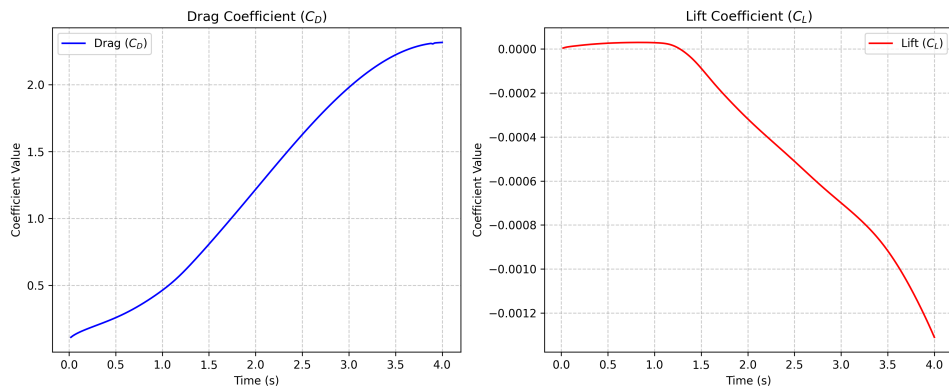
(a) Cylinder: Coarse mesh



(b) Cylinder: Fine mesh



(c) Parallelepiped: Coarse mesh



(d) Parallelepiped: Fine mesh

Figure 10: Drag and Lift coefficients (Cylinder and Parallelepiped) for the 3D-3 unsteady test case.

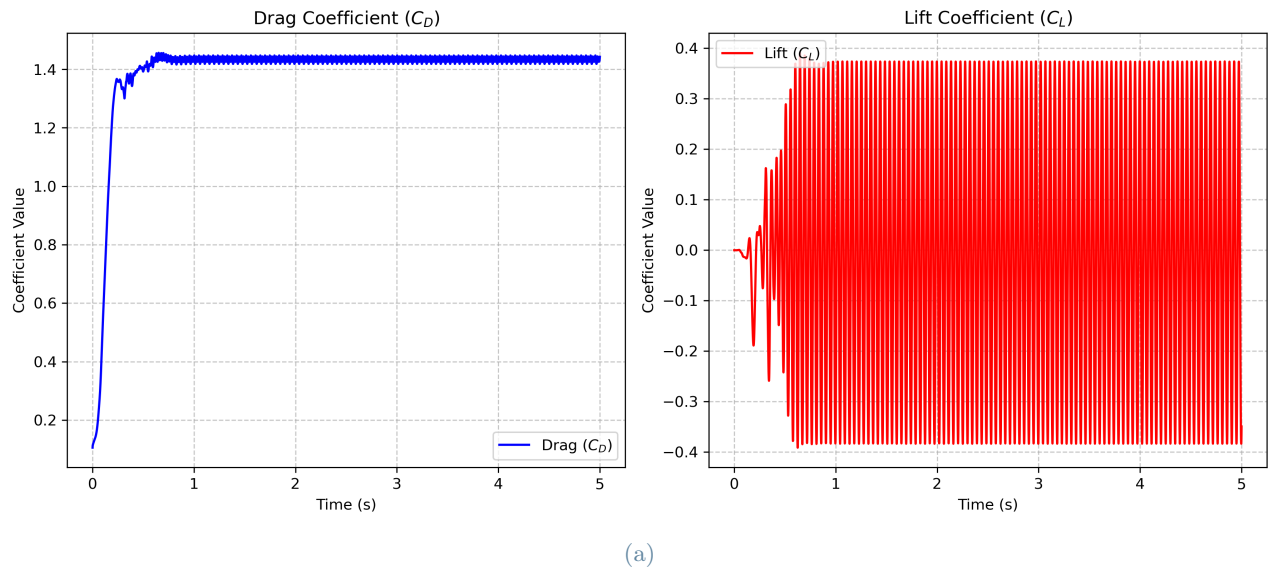


Figure 11: Drag and Lift coefficients for the $Re = 1000$ Chorin - Temam test case.

B. Simulation Screenshots

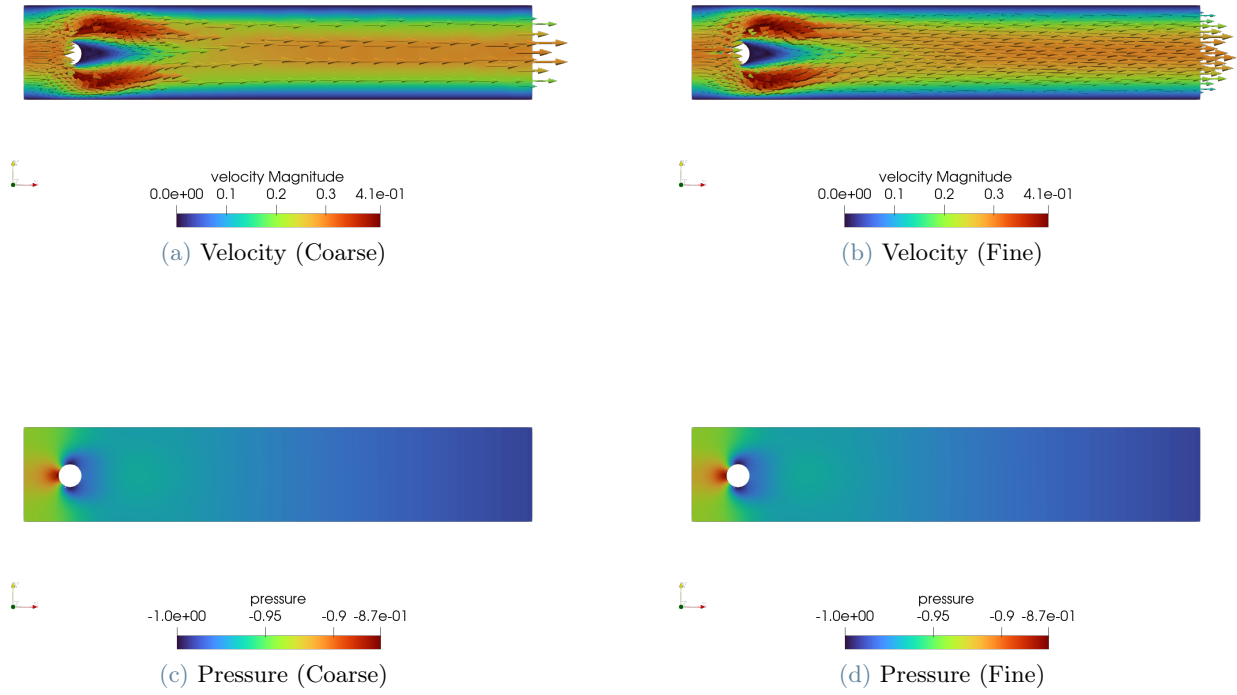


Figure 12: Instantaneous velocity and pressure fields at $t = 5s$ for the coarse (left) and fine (right) meshes in the 2D-1 steady test case.

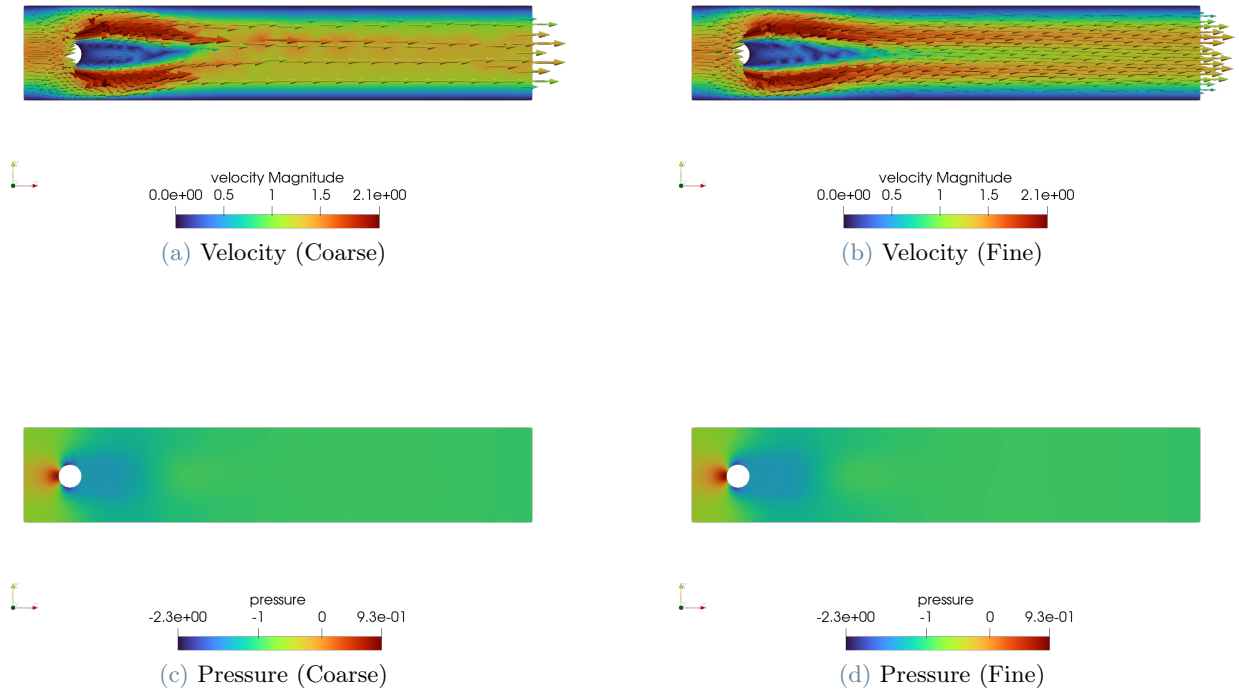


Figure 13: Instantaneous velocity and pressure fields at $t = 5s$ for the coarse (left) and fine (right) meshes in the 2D-2 steady test case.

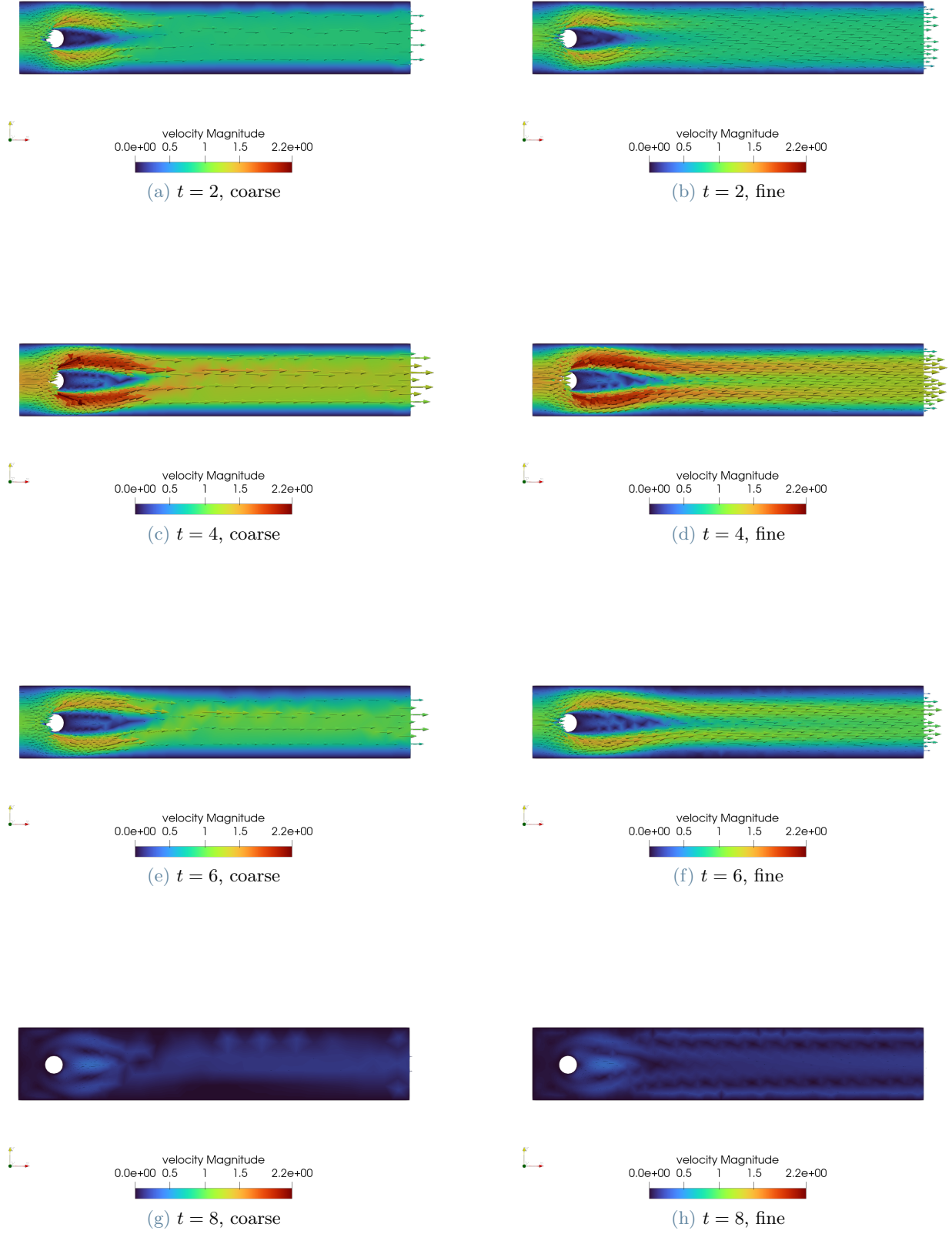


Figure 14: Instantaneous velocity fields at selected times for the coarse (left) and fine (right) meshes in the 2D-3 unsteady test case.

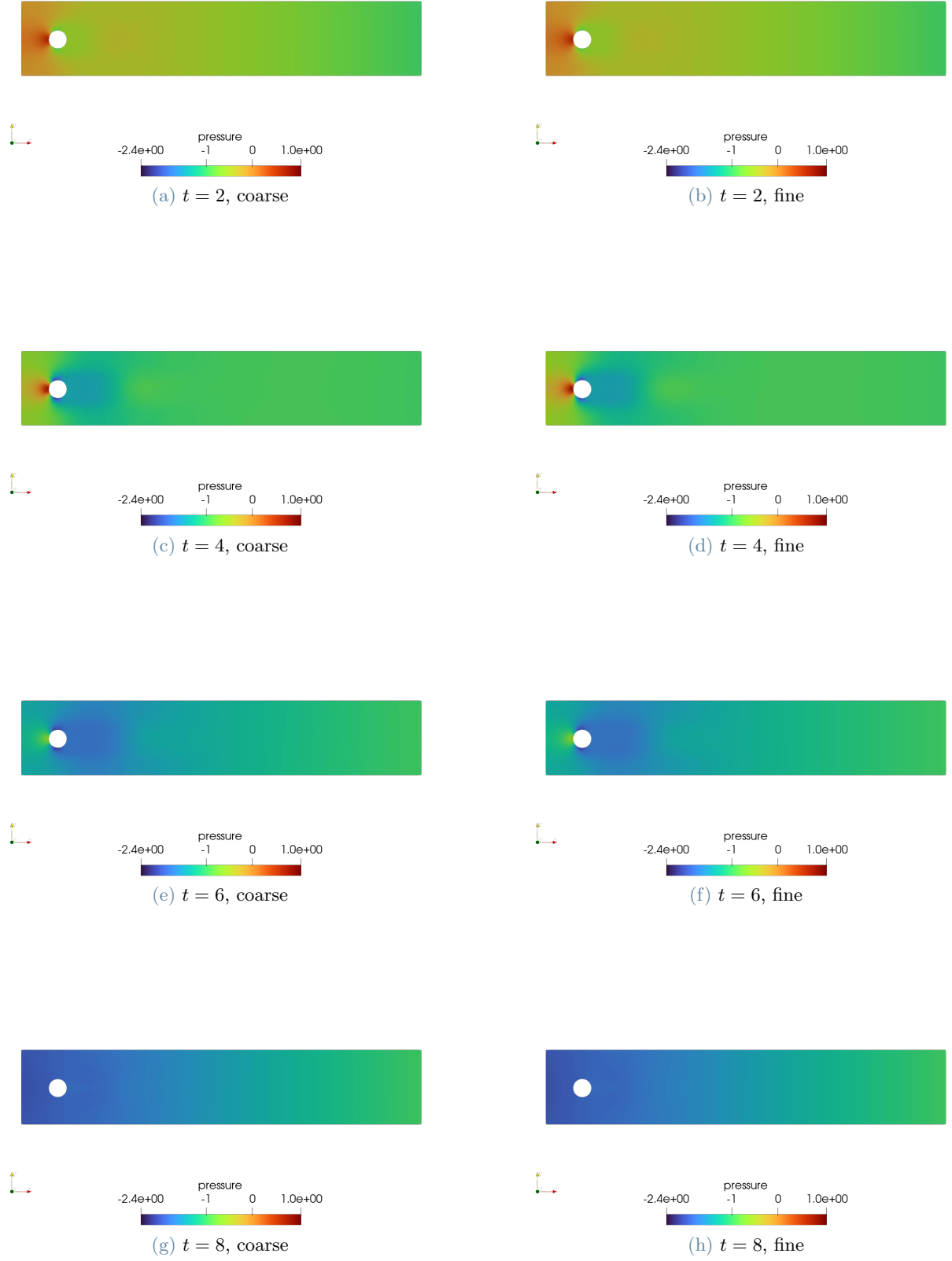
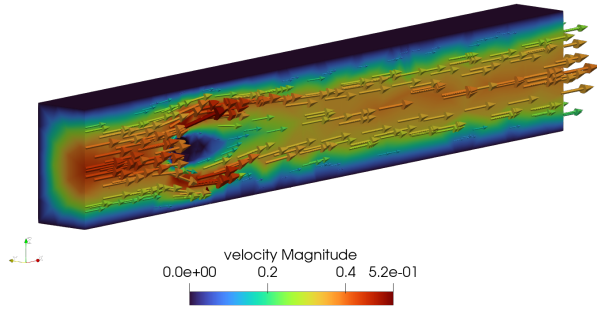
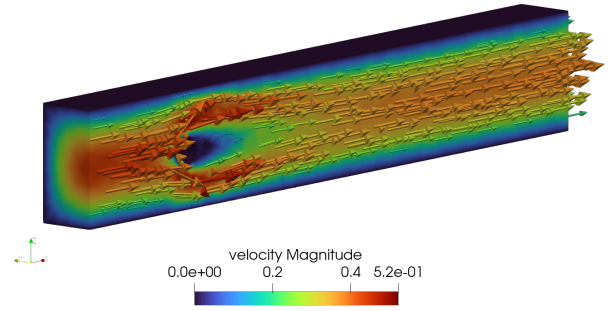


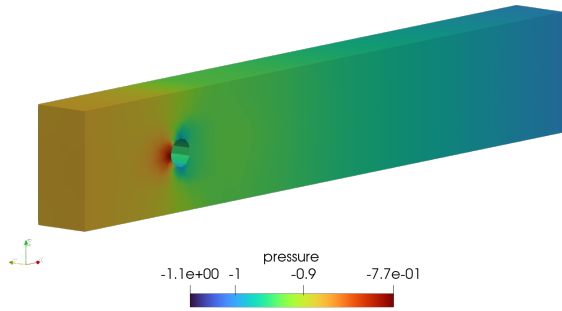
Figure 15: Instantaneous pressure fields at selected times for the coarse (left) and fine (right) meshes in the 2D-3 unsteady test case.



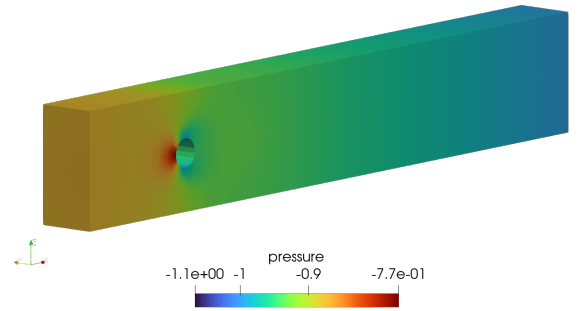
(a) Cylinder Velocity (Coarse)



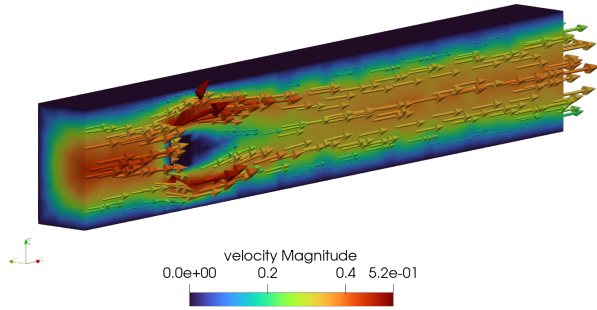
(b) Cylinder Velocity (Fine)



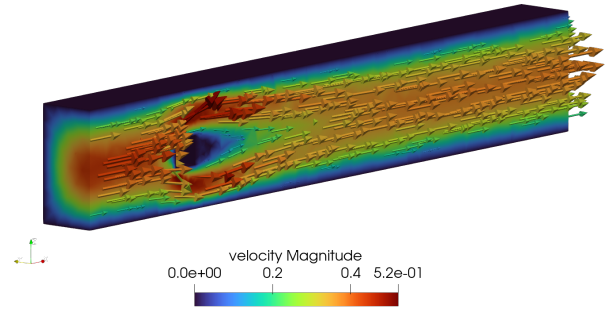
(c) Cylinder Pressure (Coarse)



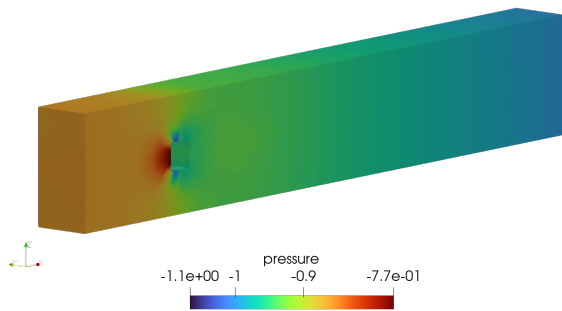
(d) Cylinder Pressure (Fine)



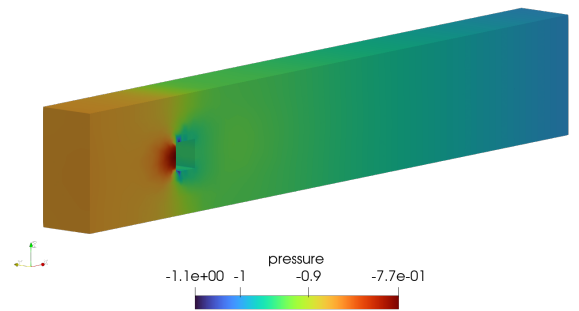
(e) Parallelepiped Velocity (Coarse)



(f) Parallelepiped Velocity (Fine)

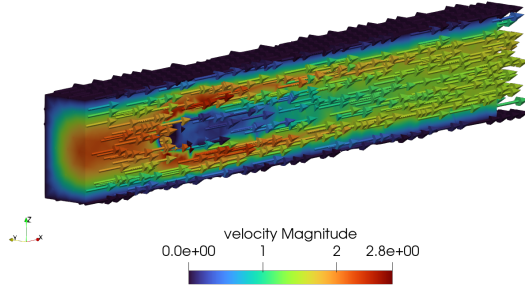


(g) Parallelepiped Pressure (Coarse)

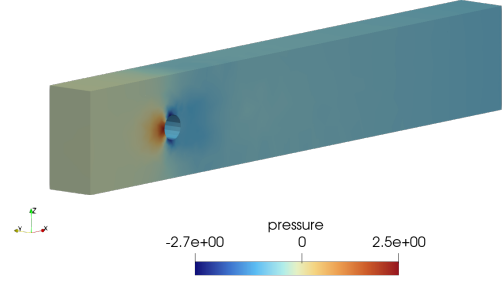


(h) Parallelepiped Pressure (Fine)

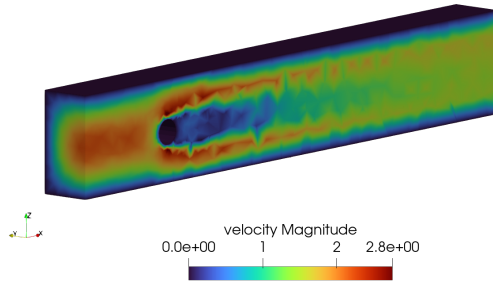
Figure 16: Instantaneous velocity and pressure fields for square and circular section obstacles at $t = 4s$ for the coarse (left) and fine (right) meshes in the 3D-1 steady test case.



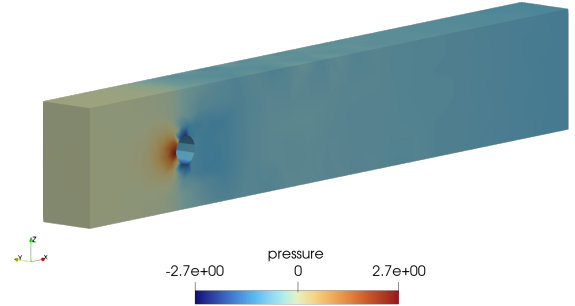
(a) Cylinder Velocity (Fine)



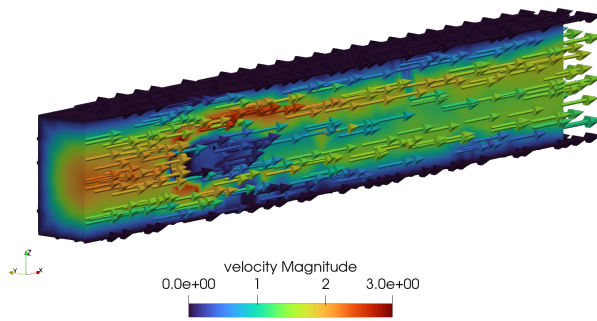
(b) Cylinder Pressure (Fine)



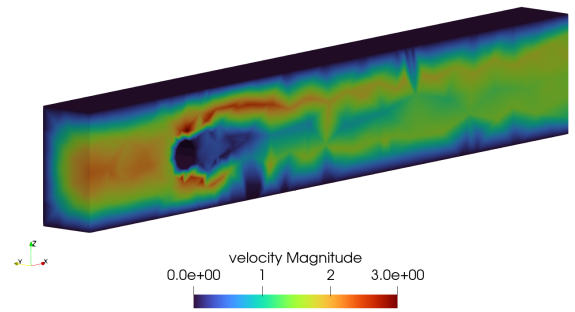
(c) Cylinder Velocity (Fine)



(d) Cylinder Pressure (Coarse)

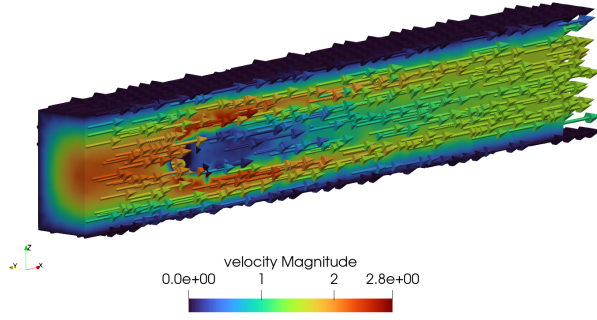


(e) Cylinder Velocity (Coarse)

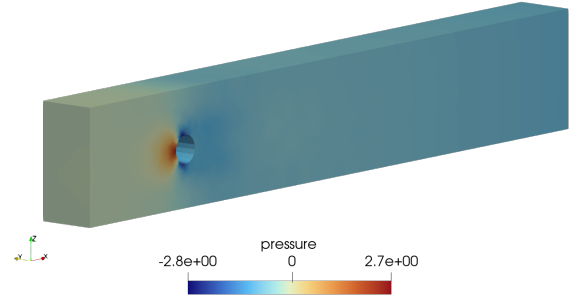


(f) Cylinder Velocity (Coarse)

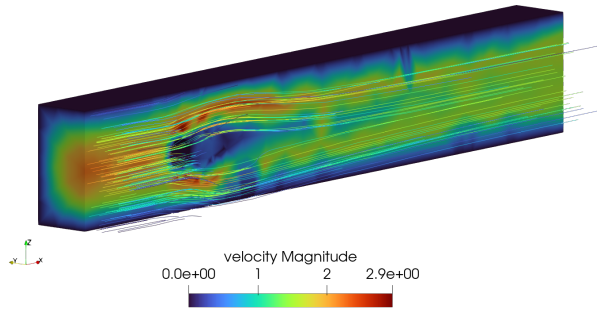
Figure 17: Selection of snapshots for the the 3D-2 steady test case.



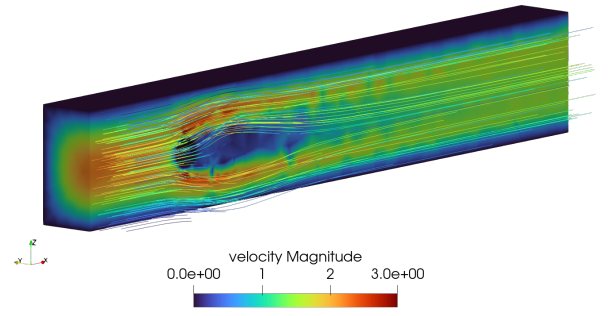
(a) Cylinder Glyph (Fine)



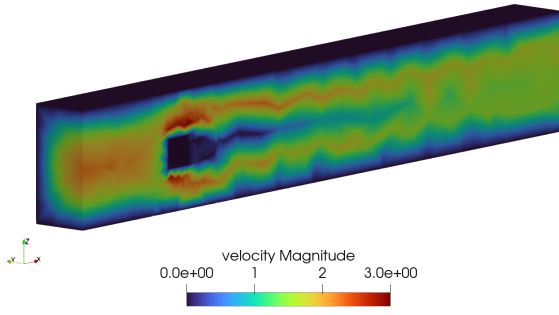
(b) Cylinder Pressure (Fine)



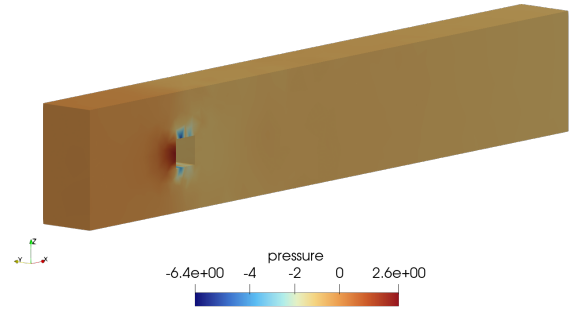
(c) Cylinder Stream Tracer (Coarse)



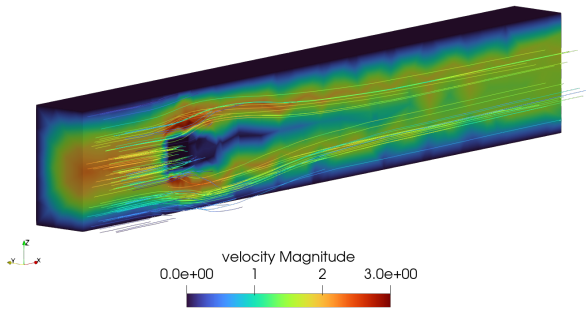
(d) Cylinder Stream Tracer (Fine)



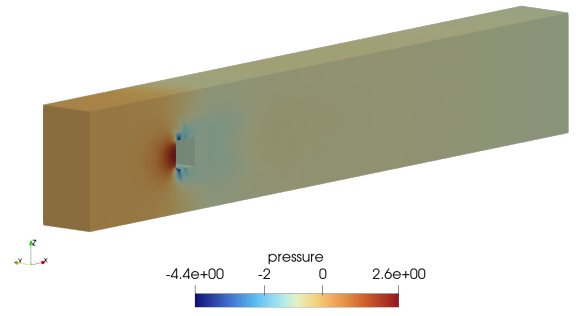
(e) Parallelepiped Velocity (Fine)



(f) Parallelepiped Pressure (Coarse)



(g) Parallelepiped Stream Tracer (Fine)



(h) Parallelepiped Pressure (Fine)

Figure 18: Selection of snapshots for the the 3D-3 unsteady test case.

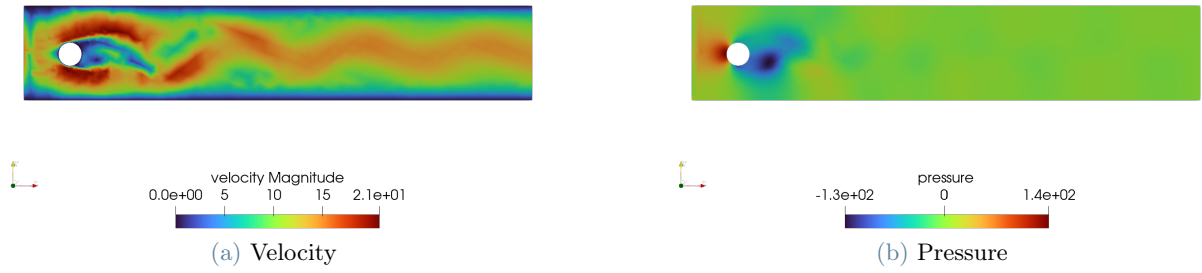


Figure 19: Instantaneous velocity and pressure fields at $t = 5s$ on the fine mesh for $Re = 1000$ Chorin - Temam test case.